IU Digital de Antioquia
Tecnología en Desarrollo de Software · Ingeniería de Software

Autos Colombia

Sistema de gestión de parqueadero por mensualidad

Iteración 2: Gestión de usuarios y gestión de celdas

Presentado por:

David Santiago Cantor Torres

10 de noviembre de 2026

Contenido

1. Introducción

2. Entregables de la iteración 2

    2.1 Requerimientos funcionales y no funcionales

    2.2 Diagramas de casos de uso de gestión de usuarios y de celdas

    2.3 Historias de usuario

    2.4 Mockups de las historias de usuario

    2.5 Diagramas de secuencia de las iteraciones 1 y 2

    2.6 Diagrama de componentes de las iteraciones 1 y 2

    2.7 Plan de pruebas resumido

    2.8 Código fuente y repositorio

3. Conclusiones

4. Referencias

1. Introducción

El parqueadero **Autos Colombia** presta el servicio de estacionamiento por mensualidad. En la **iteración 1** se construyó la gestión de la entrada y salida de vehículos: el operador registra cada movimiento con la placa, el sistema valida la mensualidad, asigna y libera celdas y deja trazabilidad de quién, cuándo y dónde. Para que esa iteración funcionara de punta a punta se incluyó solo un soporte mínimo de usuarios y celdas: dos usuarios fijos, registro de vehículos con su propietario y la opción de poner una celda en mantenimiento.

Este documento presenta la **iteración 2: gestión de usuarios y gestión de celdas**. En el sistema existen dos clases de usuarios, y ambas se gestionan en esta iteración:

Tipo de usuario	Qué se gestiona
Usuarios del sistema (empleados)	Administradores y operadores que inician sesión: creación, edición de datos y rol, activación y desactivación, restablecimiento y cambio de contraseña, con reglas que impiden dejar el sistema sin administrador.
Clientes (usuarios del servicio)	Personas que pagan la mensualidad: registro, consulta, edición, activación y desactivación, y asociación de sus vehículos. Un cliente inactivo no puede ingresar vehículos.

La **gestión de celdas** pasa de un simple cambio de estado a una administración completa: crear celdas individuales o por rangos, editar código y tipo, ponerlas en mantenimiento indicando el motivo, habilitarlas, eliminar las que nunca se usaron y consultar su historial de uso.

Iteración	Alcance	Estado
Iteración 1	Entrada y salida de vehículos, asignación y liberación de celdas, novedades, vehículos dentro e historial.	Entregada
Iteración 2	Gestión de usuarios del sistema, de clientes y de celdas.	Desarrollada (este documento)
Iteración 3	Gestión de pagos de mensualidades.	Planeada

La solución mantiene la arquitectura de la iteración 1 (aplicación web en **Python 3 con Flask** y **SQLite**, organizada en controladores, servicios, repositorios y modelo de dominio). Se agregaron tres servicios (`UsuarioService`, `ClienteService` y `CeldaService`), sus controladores y vistas, y la base de datos de la iteración 1 se actualiza automáticamente sin perder información. El documento recorre los entregables solicitados: requerimientos, casos de uso, historias de usuario, mockups, diagramas de secuencia de ambas iteraciones, diagrama de componentes, plan de pruebas y repositorio.

2. Entregables de la iteración 2

2.1 Requerimientos funcionales y no funcionales

Actores

Actor	Participación en la iteración 2
Operador	Empleado de portería. Consulta clientes y celdas (mapa, listado, detalle e historial) y cambia su propia contraseña.
Administrador	Hereda lo del operador. Gestiona los usuarios del sistema, los clientes y sus vehículos, y las celdas.

Requerimientos funcionales · Gestión de usuarios del sistema

ID	Nombre	Descripción	Prioridad
RF16	Crear usuario del sistema	El administrador debe poder crear un usuario con nombre completo, nombre de usuario, rol (Administrador u Operador) y contraseña con confirmación. El usuario queda activo.	Alta
RF17	Validar credenciales	El nombre de usuario debe ser único, de 4 a 20 caracteres (minúsculas, números, punto o guion bajo) y empezar por letra. La contraseña debe tener mínimo 8 caracteres con letras y números y coincidir con su confirmación.	Alta
RF18	Editar usuario	El administrador debe poder modificar el nombre y el rol de un usuario. El nombre de usuario no se puede cambiar.	Media
RF19	Activar o desactivar usuario	El administrador debe poder desactivar un usuario (no podrá iniciar sesión) y volver a activarlo. Los usuarios no se eliminan para conservar la trazabilidad.	Alta
RF20	Proteger a los administradores	Un administrador no puede desactivarse ni quitarse el rol a sí mismo, y el sistema debe conservar siempre al menos un administrador activo.	Alta
RF21	Restablecer contraseña	El administrador debe poder asignar una nueva contraseña a un usuario que la olvidó.	Media
RF22	Cambiar mi contraseña	Todo usuario debe poder cambiar su contraseña indicando la actual; la nueva debe ser diferente y cumplir la política de RF17.	Media
RF23	Consultar usuarios	El sistema debe listar los usuarios con nombre, usuario, rol, estado y número de movimientos registrados, con búsqueda e indicadores de usuarios activos por rol.	Media

Requerimientos funcionales · Gestión de clientes

ID	Nombre	Descripción	Prioridad
RF24	Registrar cliente	El administrador debe poder registrar un cliente con documento, nombre completo, teléfono y correo.	Alta
RF25	Validar datos del cliente	El documento debe ser único y tener de 5 a 15 dígitos; el teléfono de 7 a 10 dígitos y el correo un formato válido (ambos opcionales). Los datos se normalizan: sin puntos ni espacios en el documento, nombre sin espacios sobrantes y correo en minúsculas.	Alta
RF26	Editar cliente	El administrador debe poder modificar los datos de un cliente sin repetir el documento de otro.	Media
RF27	Consultar clientes	El sistema debe listar los clientes con búsqueda, número de vehículos, estado de sus mensualidades y vehículos dentro, y mostrar el detalle de cada cliente con sus vehículos, vigencia de la mensualidad y ubicación.	Media
RF28	Activar o desactivar cliente	El administrador debe poder desactivar un cliente. Sus vehículos no podrán ingresar ni se le podrán asociar vehículos nuevos. No se puede desactivar un cliente con un vehículo dentro.	Alta
RF29	Asociar vehículo a cliente	Desde el detalle de un cliente activo, el administrador debe poder registrarle un vehículo con su mensualidad (amplía RF14 de la iteración 1).	Media

Requerimientos funcionales · Gestión de celdas

ID	Nombre	Descripción	Prioridad
RF30	Crear celda	El administrador debe poder crear una celda con código único (formato `A-09`: letra(s), guion y número) y tipo de vehículo (carro o moto). La celda queda LIBRE.	Alta
RF31	Crear celdas por rango	El administrador debe poder crear varias celdas indicando zona, número inicial y final (máximo 50). Si algún código ya existe, no se crea ninguna.	Media
RF32	Editar celda	El administrador debe poder cambiar el código y el tipo de una celda que no esté ocupada, sin repetir códigos.	Media
RF33	Mantenimiento de celda	El administrador debe poder poner en mantenimiento una celda libre indicando el motivo, y habilitarla después. Una celda en mantenimiento no se asigna a ningún ingreso. *(Amplía RF15.)*	Alta
RF34	Eliminar celda	El administrador debe poder eliminar una celda que no esté ocupada y que nunca se haya usado; si tiene historial, el sistema indica usar mantenimiento.	Baja

RF35	Consultar celdas	El sistema debe mostrar el resumen por tipo (libres, ocupadas, mantenimiento), el mapa y un listado filtrable con estado, placa u observación y número de usos, y el detalle de cada celda con sus últimos usos.	Media

Requerimientos no funcionales

ID	Tipo	Descripción
RNF10	Seguridad · autorización	Las operaciones de gestión (crear, editar, activar, desactivar, eliminar) solo están disponibles para el rol Administrador, validado en el servidor; si un operador intenta acceder por URL es redirigido sin ejecutar la acción. Las consultas están disponibles para ambos roles.
RNF11	Seguridad · contraseñas	Las contraseñas se almacenan solo como hash (scrypt de Werkzeug), cumplen una política mínima (8 caracteres, letras y números) y nunca se muestran. Un usuario desactivado recibe el mismo mensaje que unas credenciales incorrectas.
RNF12	Integridad de datos	El nombre de usuario, el documento del cliente y el código de la celda son únicos en la base de datos (UNIQUE) y en la capa de servicios. La creación de celdas por rango es atómica (todo o nada).
RNF13	Trazabilidad	Usuarios y clientes se desactivan en lugar de borrarse, y las celdas con historial no se eliminan, para conservar las entradas, salidas y novedades asociadas.
RNF14	Usabilidad	Las acciones que restringen o borran (desactivar, eliminar) piden confirmación; los formularios muestran las reglas, conservan lo digitado ante un error y los mensajes explican la causa en español. Los listados tienen búsqueda inmediata.
RNF15	Compatibilidad de datos	Al iniciar, la aplicación actualiza automáticamente una base de datos creada en la iteración 1 (nuevas columnas) sin perder información.
RNF16	Mantenibilidad	Cada área nueva tiene su servicio, controlador y pruebas automatizadas; las reglas de negocio se prueban sin interfaz.
RNF17	Rendimiento	Los listados de clientes, usuarios y celdas deben cargar en menos de 1 segundo con hasta 500 clientes y 200 celdas.

Reglas de negocio

ID	Regla
RN05	Siempre debe existir al menos un administrador activo; nadie puede desactivarse ni quitarse el rol de administrador a sí mismo.
RN06	Un usuario del sistema inactivo no puede iniciar sesión.
RN07	Un cliente inactivo no puede ingresar vehículos ni recibir vehículos nuevos; no se puede desactivar un cliente con un vehículo dentro.
RN08	Una celda OCUPADA no se puede editar, poner en mantenimiento ni eliminar.
RN09	Solo se eliminan celdas sin historial de uso; las demás se retiran poniéndolas en mantenimiento.

RN10	Toda celda en mantenimiento registra su motivo.

2.2 Diagramas de casos de uso

Los diagramas se elaboraron en **StarUML** (archivo `docs/AutosColombia.mdj`, paquete *Autos Colombia - Iteración 2*). Como en la iteración 1, el Administrador hereda los casos de uso del Operador. Las relaciones «include» representan pasos obligatorios que el sistema siempre ejecuta (validar datos, cifrar la contraseña, verificar que la celda no esté ocupada) y las «extend» comportamientos opcionales (asociar un vehículo al registrar un cliente, crear celdas por rango).

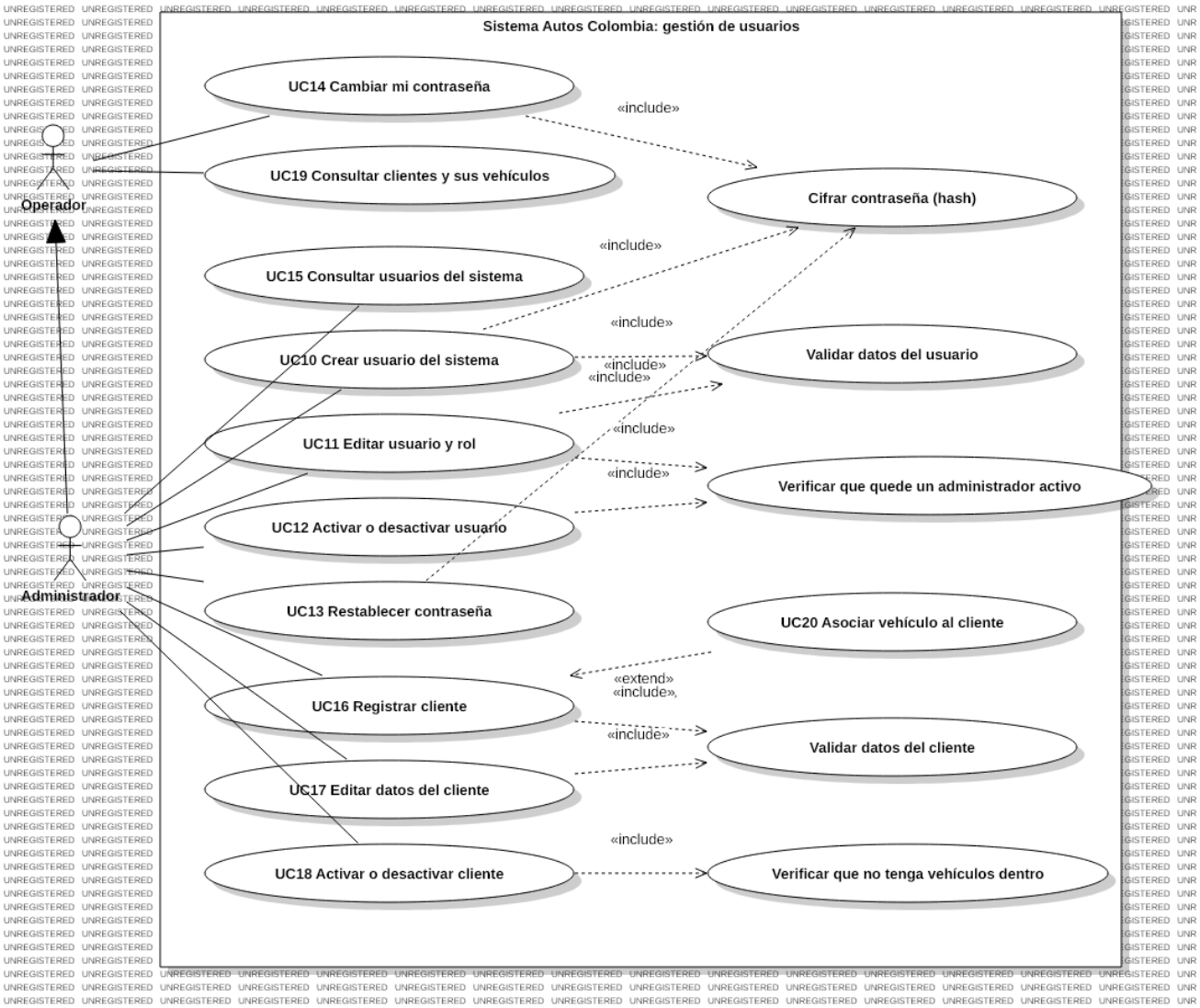


Figura 1. Diagrama de casos de uso de gestión de usuarios: usuarios del sistema y clientes (StarUML).

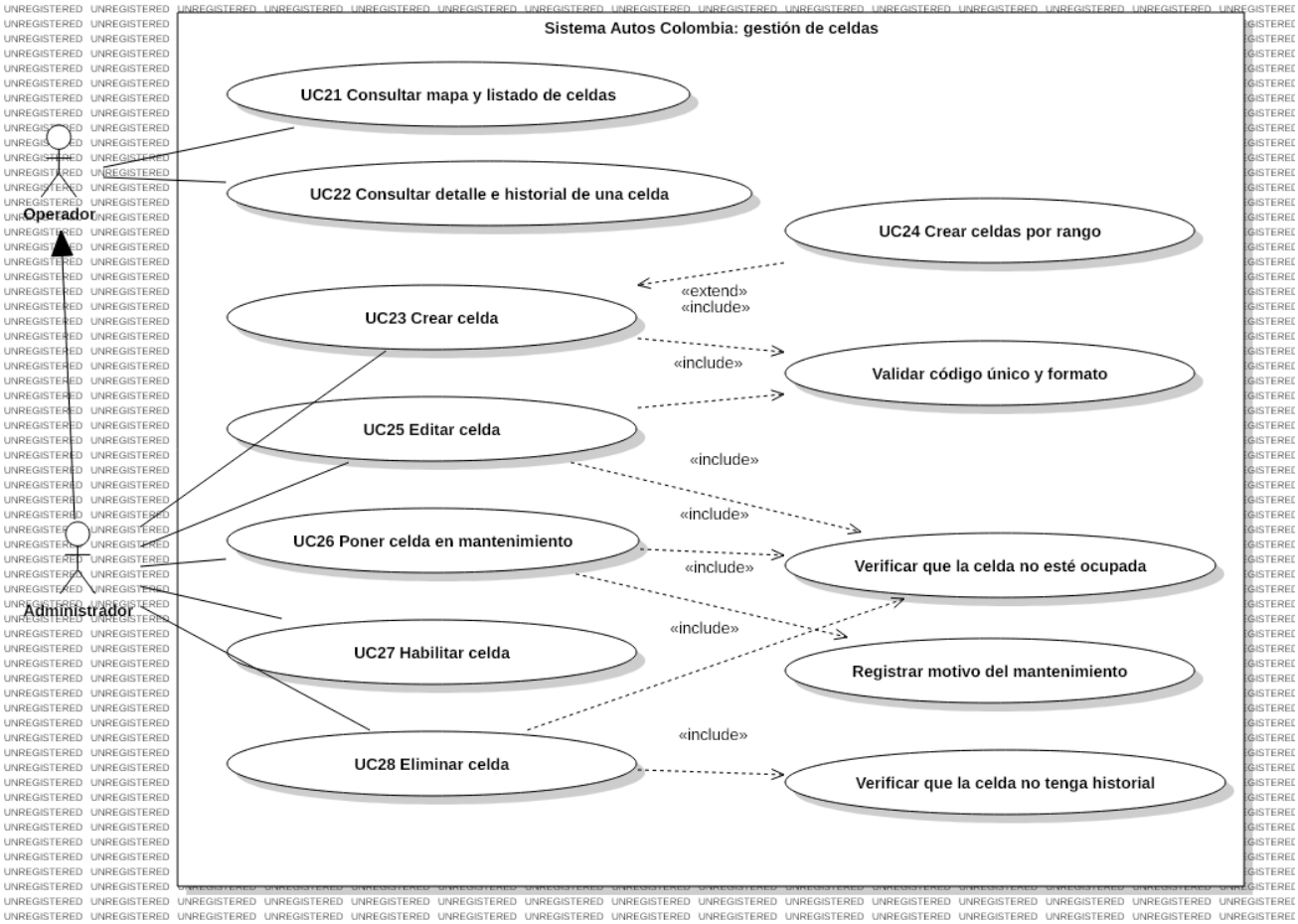


Figura 2. Diagrama de casos de uso de gestión de celdas (StarUML).

Especificación UC12 · Activar o desactivar usuario

Actor	Administrador
Precondición	El administrador inició sesión y el usuario existe.
Flujo principal	1. El administrador abre “Usuarios del sistema” y pulsa “Desactivar” sobre un usuario. 2. El sistema pide confirmación. 3. El sistema verifica que no sea su propio usuario y que quede otro administrador activo (include). 4. Marca el usuario como inactivo y muestra el mensaje “ya no podrá iniciar sesión”.
Flujos alternos	3a. Es el mismo administrador → “No puede desactivar su propio usuario”. 3b. Es el último administrador activo → “Debe existir al menos un administrador activo”. 1a. El usuario está inactivo → “Activar” lo deja activo de nuevo.
Postcondición	El usuario no puede iniciar sesión; sus registros históricos se conservan.

Especificación UC26 · Poner celda en mantenimiento

Actor	Administrador
Precondición	El administrador inició sesión.
Flujo principal	1. El administrador selecciona una celda en el mapa o el listado. 2. Escribe el motivo del mantenimiento y confirma. 3. El sistema verifica que la celda no esté ocupada (include) y registra el motivo (include). 4. La celda queda en MANTENIMIENTO y deja de ofrecerse al registrar entradas.
Flujos alternos	3a. La celda está ocupada → "registre primero la salida". 3b. Motivo con menos de 5 caracteres → se solicita el motivo. 4a. Cuando la celda se repara, el administrador la habilita (UC27) y vuelve a LIBRE.
Postcondición	La celda no se asigna a ningún vehículo hasta ser habilitada.

2.3 Historias de usuario

Formato *Como [rol] quiero [funcionalidad] para [beneficio]*, con criterios de aceptación verificables y estimación en puntos de historia (Fibonacci). La numeración continúa la de la iteración 1 (HU01–HU09).

HU10 · Crear usuarios del sistema

Como **administrador** quiero **crear cuentas para los empleados con su rol** para **que cada persona ingrese con su propio usuario y quede registrado lo que hace**.

- Ingreso nombre, usuario, rol y contraseña con confirmación; el usuario se guarda en minúsculas y queda activo.
- Si el nombre de usuario ya existe o no cumple el formato, se informa y no se crea.
- Si la contraseña tiene menos de 8 caracteres, no tiene letras y números o no coincide con la confirmación, se informa.
- La contraseña nunca se muestra ni se guarda en texto plano.

Prioridad: Alta · Estimación: 3 pts · RF16, RF17, RNF11 · UC10 · Mockups: figuras 4 y 5

HU11 · Editar usuario y su rol

Como **administrador** quiero **cambiar el nombre y el rol de un empleado** para **ajustar sus permisos cuando cambian sus funciones**.

- Puedo cambiar nombre y rol; el nombre de usuario se muestra pero no se edita.
- No puedo quitarme a mí mismo el rol de administrador.
- Si el usuario es el último administrador activo, no se le puede quitar el rol.

Prioridad: Media · Estimación: 2 pts · RF18, RF20 · UC11 · Mockup: figura 6

HU12 · Activar y desactivar usuarios

Como **administrador** quiero **desactivar a los empleados que ya no trabajan en el parqueadero** para **que no puedan ingresar sin perder el historial de lo que registraron**.

- La lista muestra rol, estado y movimientos de cada usuario, con búsqueda e indicadores de activos.
- Al desactivar se pide confirmación; el usuario desactivado no puede iniciar sesión y puede reactivarse.
- No puedo desactivarme a mí mismo ni dejar el sistema sin administradores activos.

Prioridad: Alta · Estimación: 3 pts · RF19, RF20, RF23, RN05, RN06 · UC12, UC15 · Mockup: figura 3

HU13 · Restablecer contraseña de un empleado

Como **administrador** quiero **asignar una nueva contraseña a un empleado que la olvidó** para **que pueda seguir trabajando sin crear otra cuenta**.

- Desde la edición del usuario escribo la nueva contraseña y su confirmación.
- La nueva contraseña cumple la misma política; la anterior deja de funcionar.

Prioridad: Media · Estimación: 1 pt · RF21 · UC13 · Mockup: figura 6

HU14 · Cambiar mi contraseña

Como **empleado del parqueadero** quiero **cambiar mi contraseña** para **mantener segura mi cuenta**.

- Accedo desde el ícono de llave de la barra superior.
- Debo escribir la contraseña actual correcta; la nueva debe ser diferente y cumplir la política.

Prioridad: Media · Estimación: 1 pt · RF22 · UC14 · Mockup: figura 7

HU15 · Registrar cliente

Como **administrador** quiero **registrar a los clientes de mensualidad con sus datos de contacto** para **tener su información antes de asociarles vehículos**.

- Ingreso documento, nombre, teléfono y correo; el documento se guarda sin puntos ni espacios.
- Si el documento ya existe o algún dato no tiene el formato esperado, se informa y no se registra.
- Al guardar veo el detalle del cliente con la opción de asociar vehículos.

Prioridad: Alta · Estimación: 2 pts · RF24, RF25 · UC16 · Mockup: figura 9

HU16 · Consultar y editar clientes

Como **operador o administrador** quiero **buscar un cliente y ver sus vehículos y mensualidades** para **atender sus consultas en portería**.

- El listado permite buscar por nombre, documento, teléfono o correo y muestra si las mensualidades están al día y cuántos vehículos están dentro.
- El detalle muestra los vehículos con vigencia de la mensualidad y su ubicación (dentro con la celda, o fuera).
- Solo el administrador puede editar los datos; el documento no puede repetirse.

Prioridad: Media · Estimación: 3 pts · RF26, RF27 · UC17, UC19 · Mockups: figuras 8 y 10

HU17 · Activar y desactivar clientes

Como **administrador** quiero **desactivar a un cliente que se retira del servicio** para **que sus vehículos no puedan ingresar**.

- Al desactivar se pide confirmación; los vehículos del cliente son rechazados en la entrada con el mensaje “cliente inactivo”.
- No puedo desactivar un cliente que tiene un vehículo dentro.
- Puedo reactivarlo y sus vehículos vuelven a ingresar si tienen mensualidad vigente.

Prioridad: Alta · Estimación: 2 pts · RF28, RN07 · UC18 · Mockup: figura 10

HU18 · Asociar vehículos a un cliente

Como **administrador** quiero **registrar un vehículo desde el detalle del cliente** para **no volver a escribir los datos del propietario**.

- El formulario muestra el cliente ya seleccionado y solo pide placa, tipo, marca, color y mensualidad.
- Al guardar regreso al detalle del cliente con el vehículo en la lista.
- No se pueden asociar vehículos a un cliente inactivo.

Prioridad: Media · Estimación: 2 pts · RF29 · UC20 · Mockups: figuras 10 y 11

HU19 · Crear celdas

Como **administrador** quiero **crear celdas una por una o por rangos** para **ampliar la capacidad del parqueadero sin depender de un programador**.

- Para una celda indico código y tipo; para un rango indico zona, desde, hasta y tipo, y veo una vista previa de los códigos.
- Los códigos deben tener el formato `A-09` y no repetirse; si alguno del rango existe, no se crea ninguno.
- Las celdas nuevas quedan libres y aparecen en el mapa y en la lista de la entrada.

Prioridad: Alta · Estimación: 3 pts · RF30, RF31 · UC23, UC24 · Mockups: figuras 12 y 13

HU20 · Editar celda

Como **administrador** quiero **corregir el código o el tipo de una celda** para **reflejar cambios en la demarcación del parqueadero**.

- Edito desde el detalle de la celda.
- Si la celda está ocupada el formulario se deshabilita; un código repetido se rechaza.

Prioridad: Media · Estimación: 1 pt · RF32, RN08 · UC25 · Mockup: figura 14

HU21 · Mantenimiento de celdas

Como **administrador** quiero **poner una celda en mantenimiento indicando el motivo y habilitarla después** para **no asignar espacios que no se pueden usar y saber por qué están fuera de servicio**.

- Debo escribir el motivo (mínimo 5 caracteres); se muestra en el detalle, en el listado y al pasar el cursor por el mapa.
- Una celda ocupada no se puede poner en mantenimiento.
- Al habilitarla vuelve a estar libre y disponible.

Prioridad: Alta · Estimación: 2 pts · RF33, RN08, RN10 · UC26, UC27 · Mockups: figuras 14 y 15

HU22 · Eliminar celdas creadas por error

Como **administrador** quiero **eliminar una celda que creé por error** para **mantener el mapa limpio**.

- Se pide confirmación antes de eliminar.
- Solo se eliminan celdas sin historial y no ocupadas; en otro caso se informa que debe usarse mantenimiento.

Prioridad: Baja · Estimación: 1 pt · RF34, RN09 · UC28 · Mockup: figura 16

HU23 · Consultar celdas y su historial

Como **operador** quiero **ver el estado de todas las celdas y el historial de cada una** para **responder dónde estuvo un vehículo y planear el uso del espacio**.

- Veo el resumen por tipo, el mapa con colores por estado y un listado que puedo filtrar por código, estado o placa.
- El detalle de una celda muestra sus últimos usos con placa, cliente, entrada, salida y permanencia.

Prioridad: Media · Estimación: 2 pts · RF35 · UC21, UC22 · Mockups: figuras 12 y 14

2.4 Mockups de las historias de usuario

Los mockups se construyeron como **prototipo navegable en HTML y CSS** (herramienta alternativa a Figma permitida en la actividad), con la misma línea gráfica de la iteración 1: menú lateral, barra superior con el usuario y área de contenido. Las imágenes corresponden a las pantallas del prototipo con datos de ejemplo y están en `docs/img/mockups/` (archivos 14 a 27). El menú incorpora la sección *Gestión* (Clientes, Vehículos, Celdas) y la sección *Administración* (Usuarios del sistema), visible solo para el administrador; el ícono de llave en la barra superior abre "Cambiar mi contraseña".

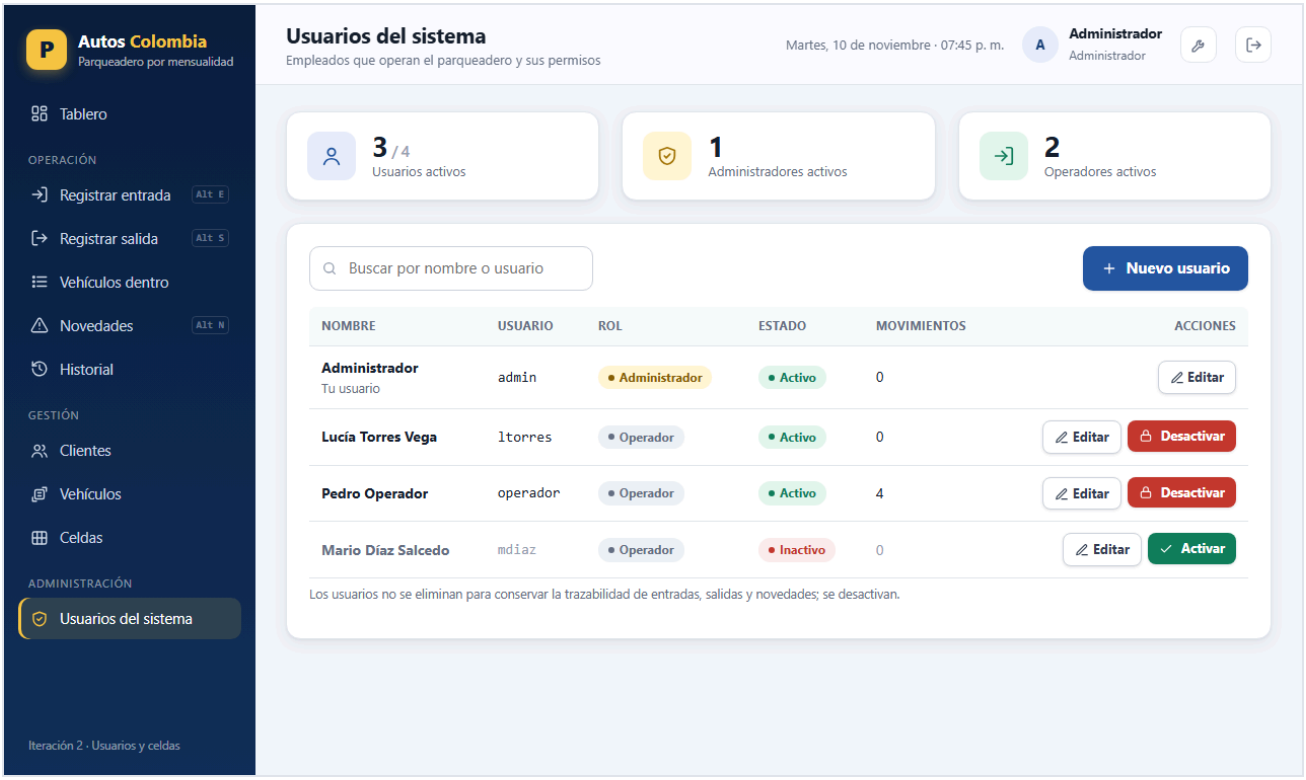


Figura 3. Usuarios del sistema: indicadores, búsqueda, edición y activación/desactivación (HU10, HU12).

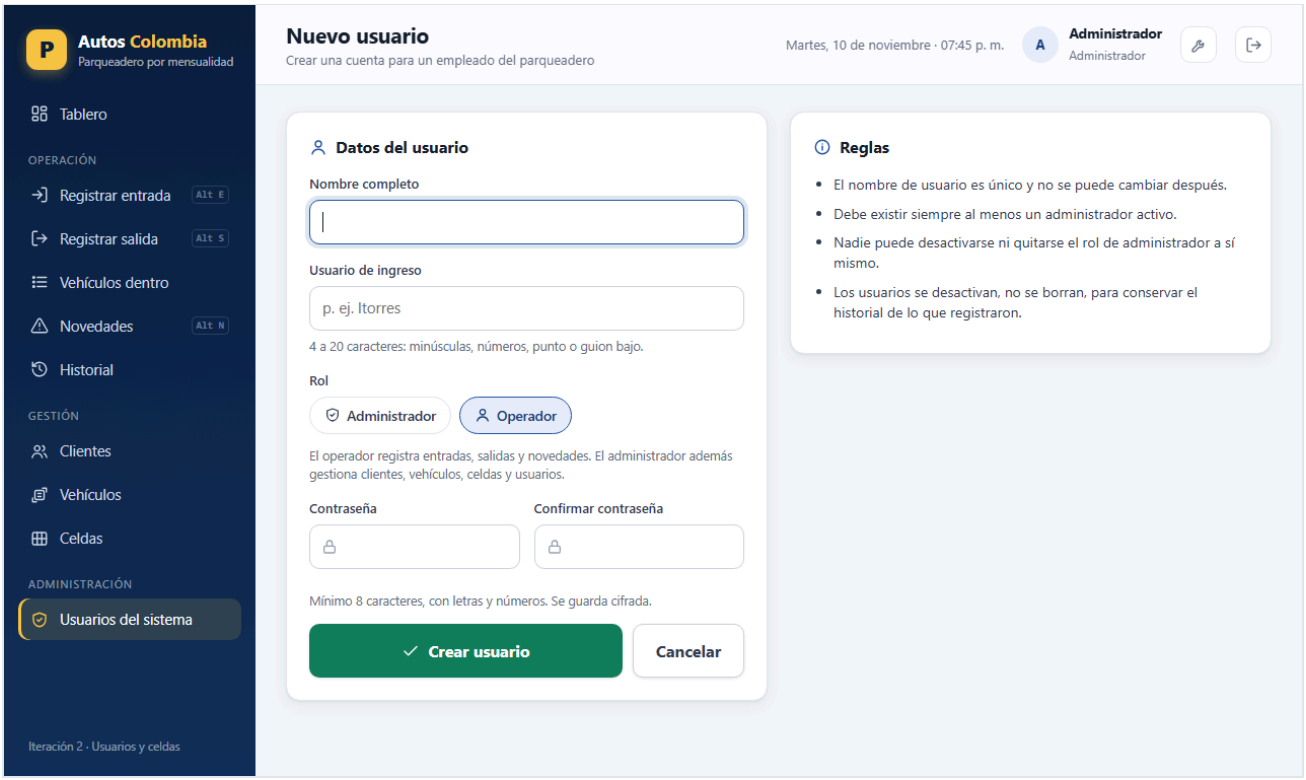


Figura 4. Crear usuario del sistema con rol y contraseña (HU10).

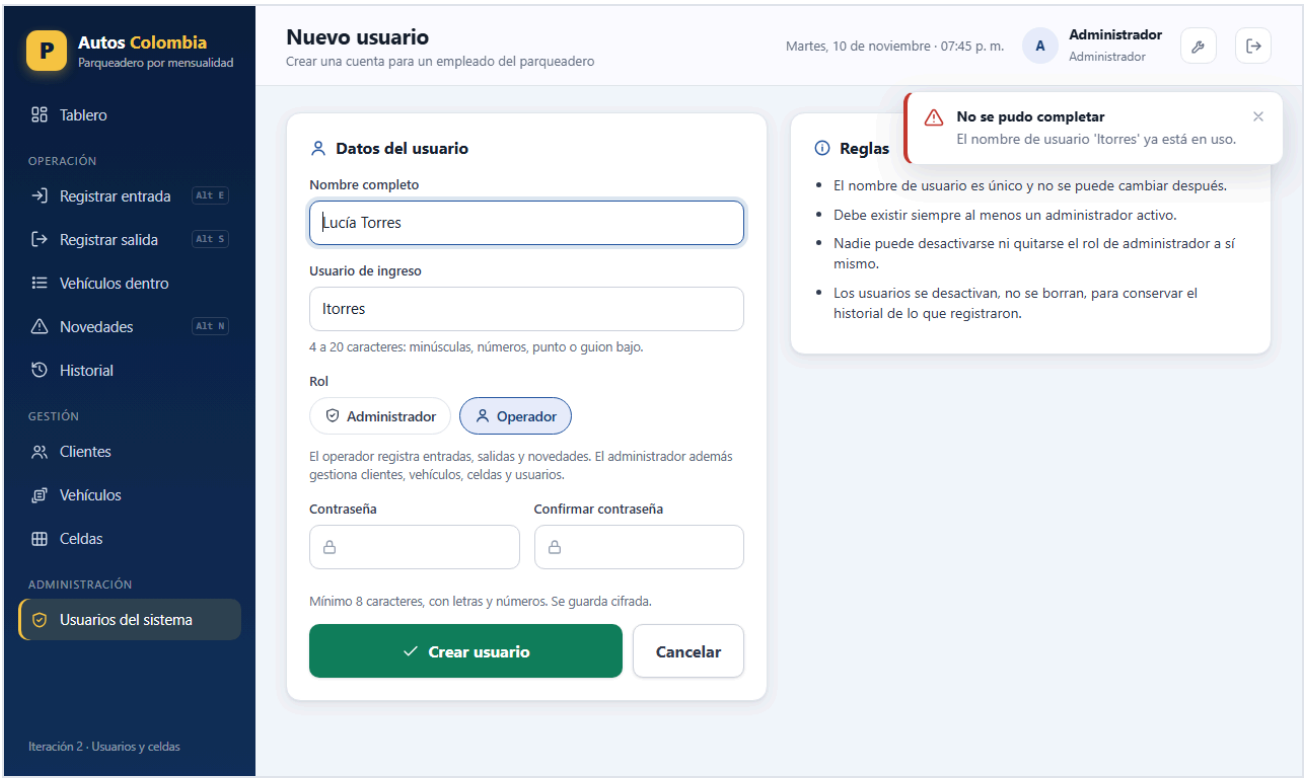


Figura 5. Creación rechazada: el nombre de usuario ya está en uso (HU10).

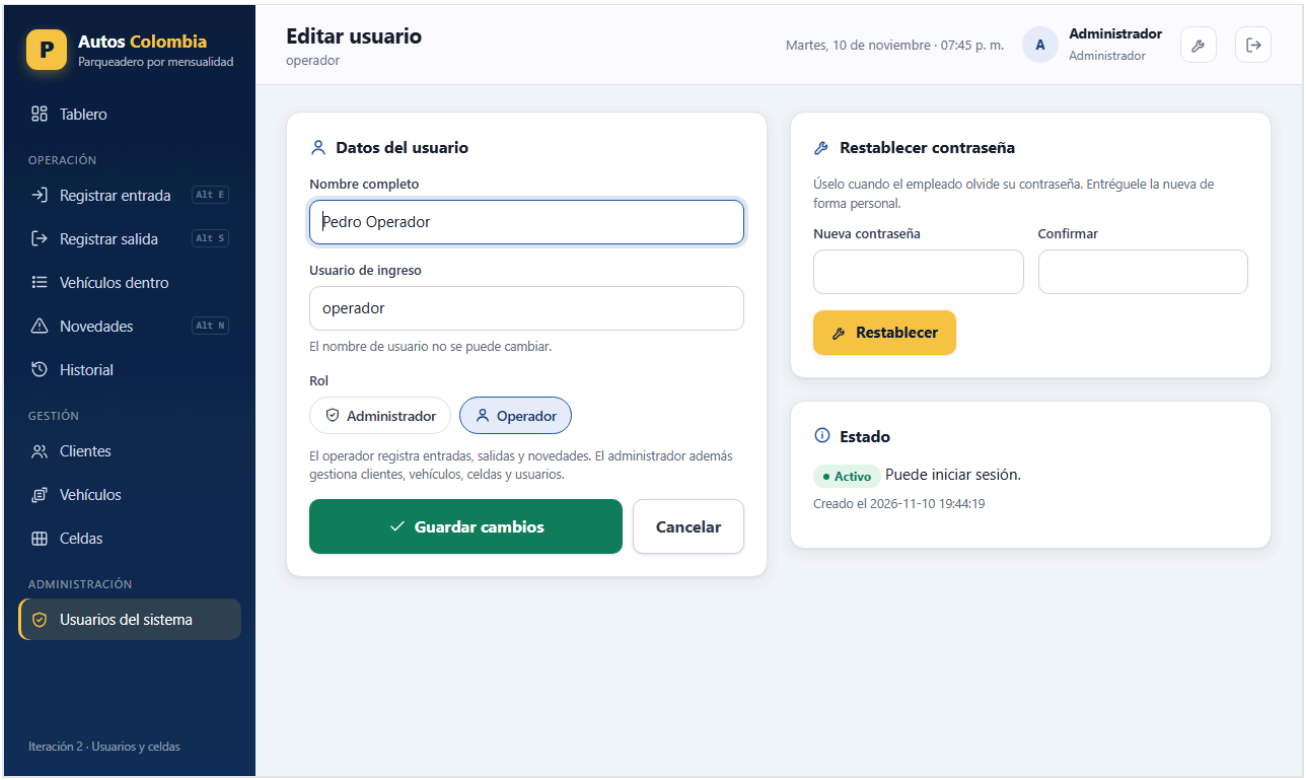


Figura 6. Editar usuario, cambiar rol y restablecer contraseña (HU11, HU13).

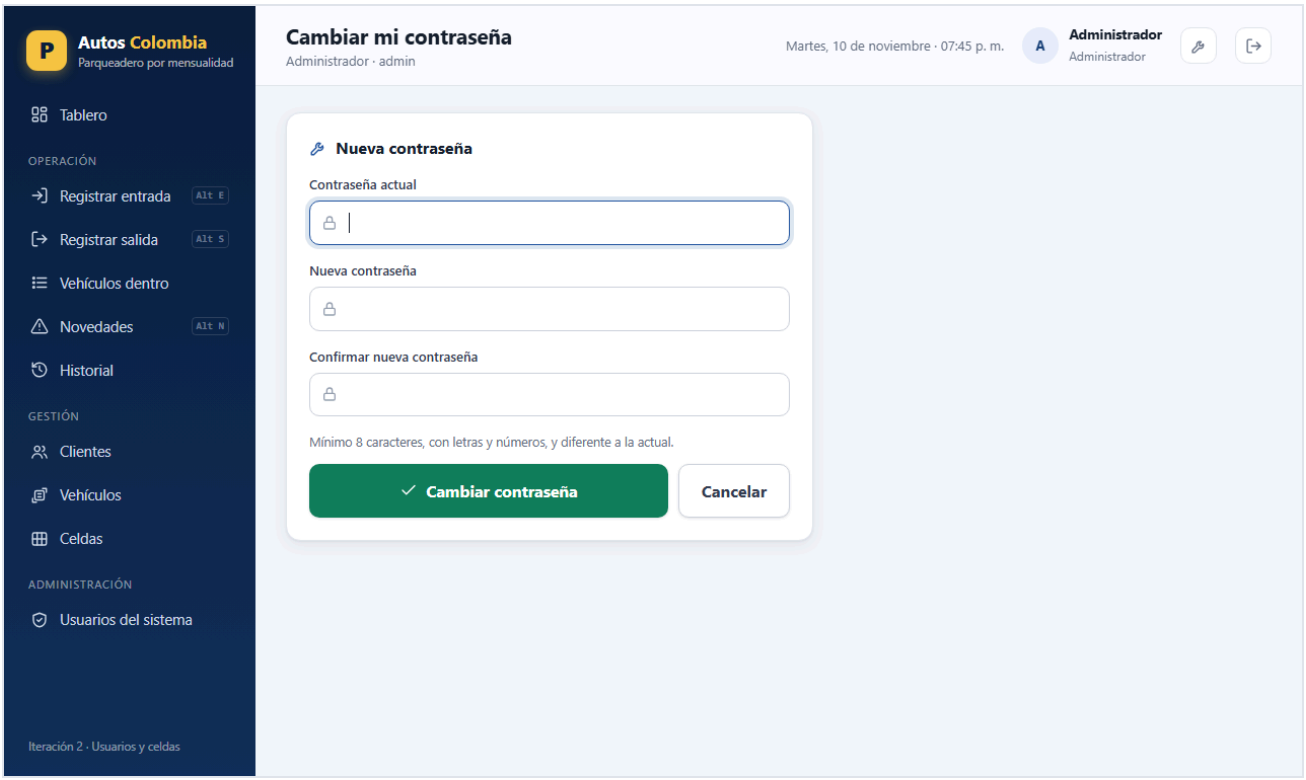


Figura 7. Cambiar mi contraseña (HU14).

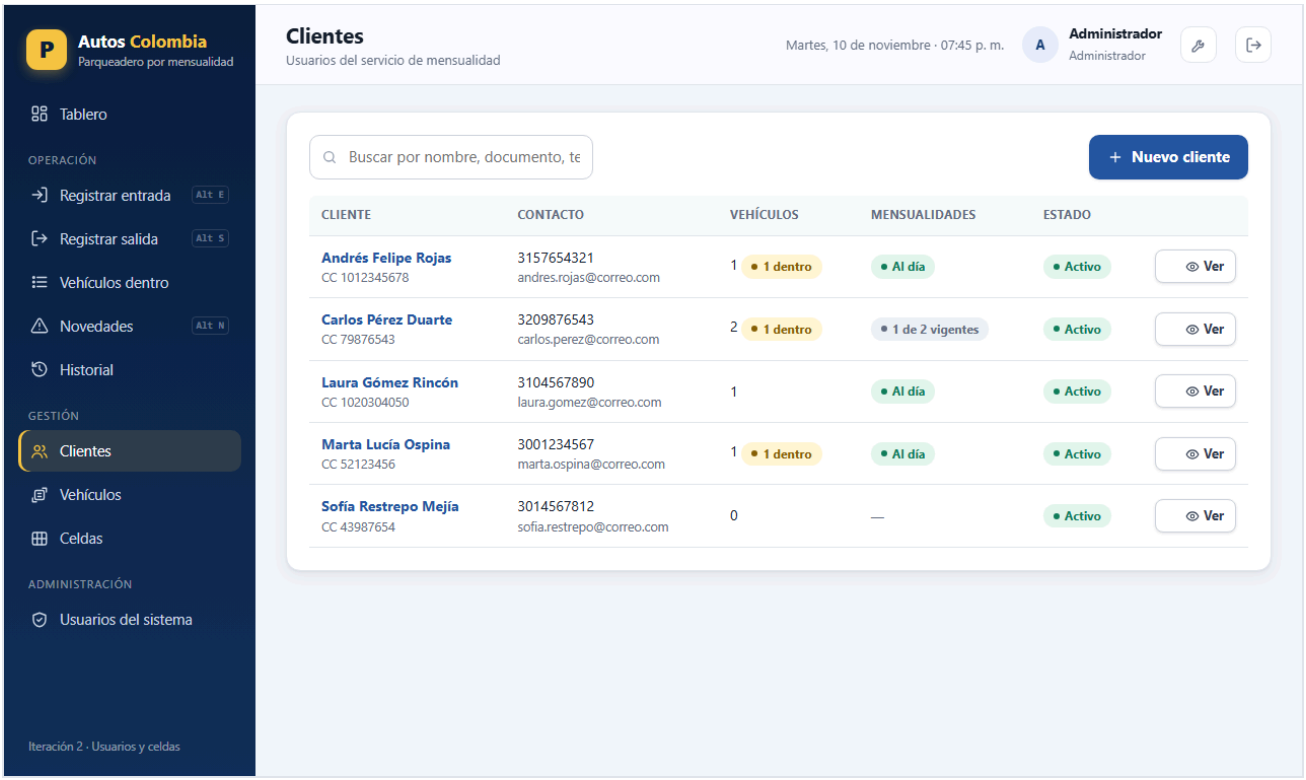


Figura 8. Listado de clientes con estado de mensualidades y vehículos dentro (HU16).

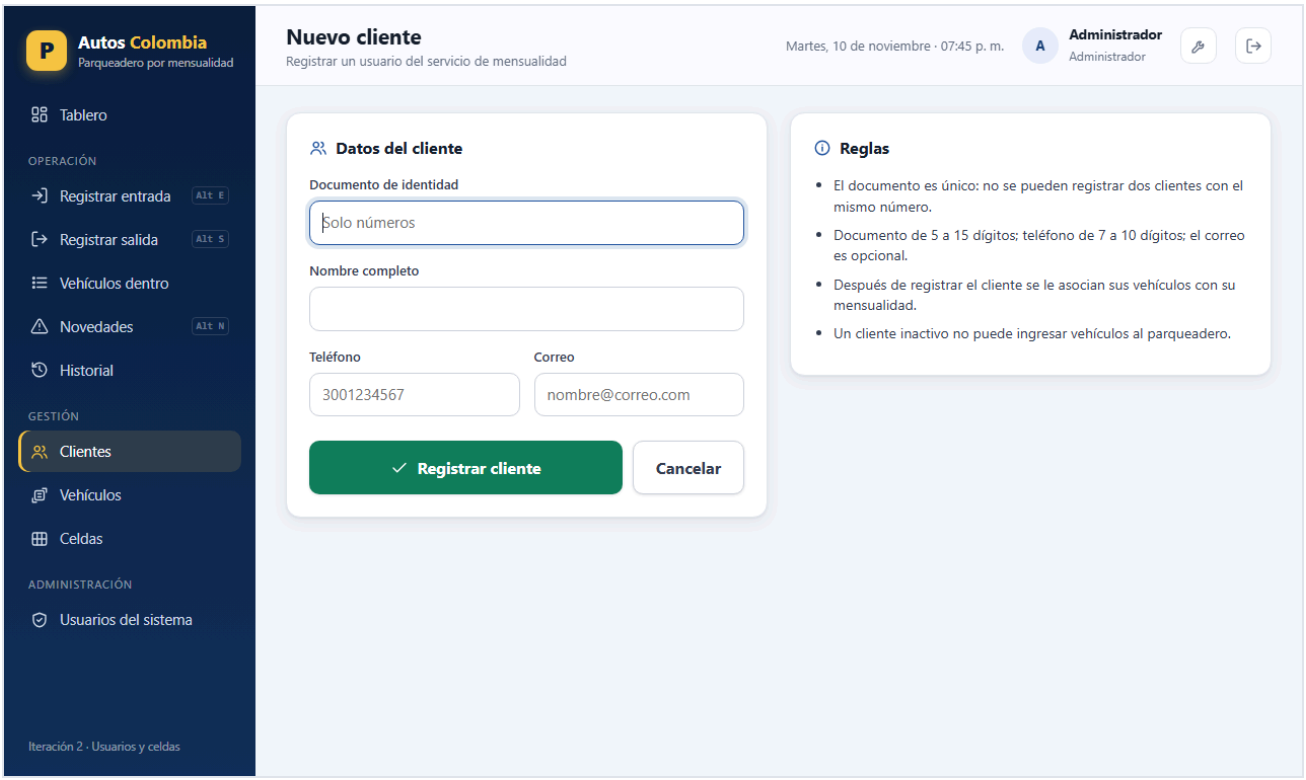


Figura 9. Registrar cliente (HU15).

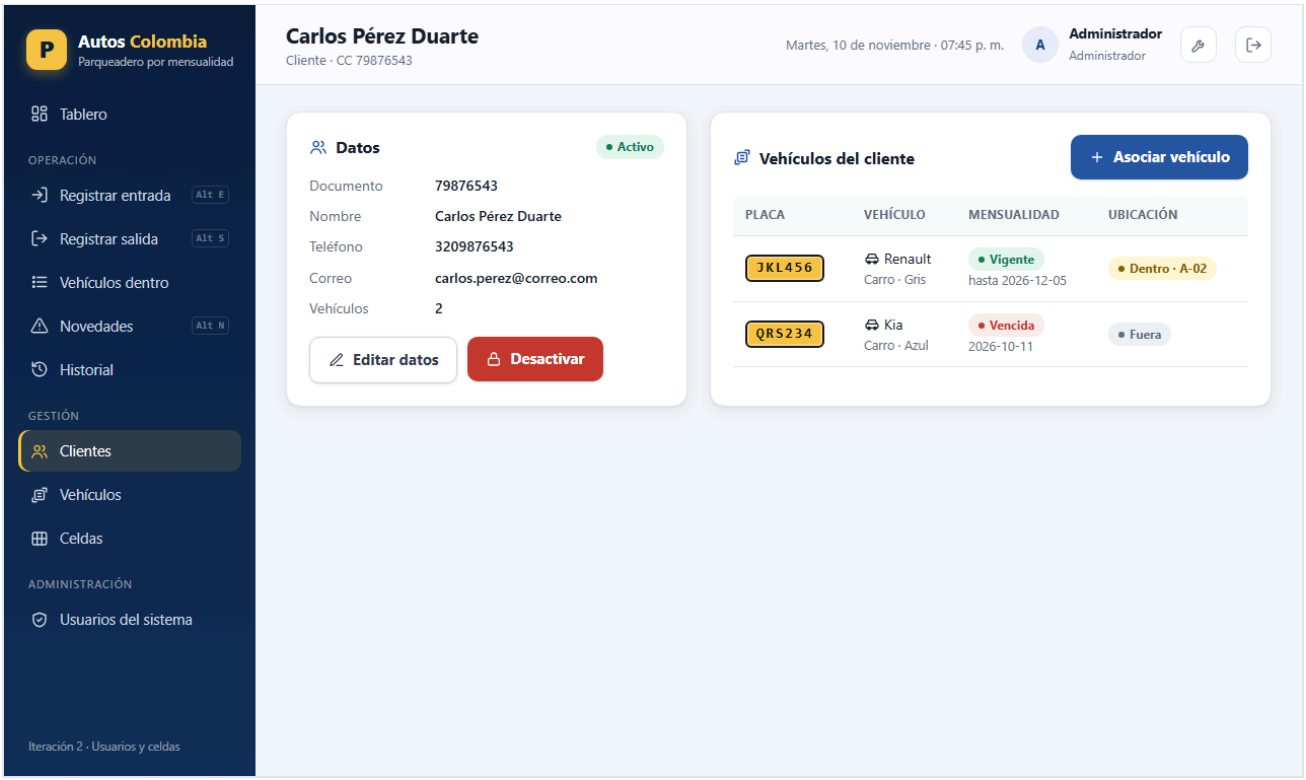


Figura 10. Detalle del cliente con sus vehículos; editar y desactivar (HU16, HU17, HU18).

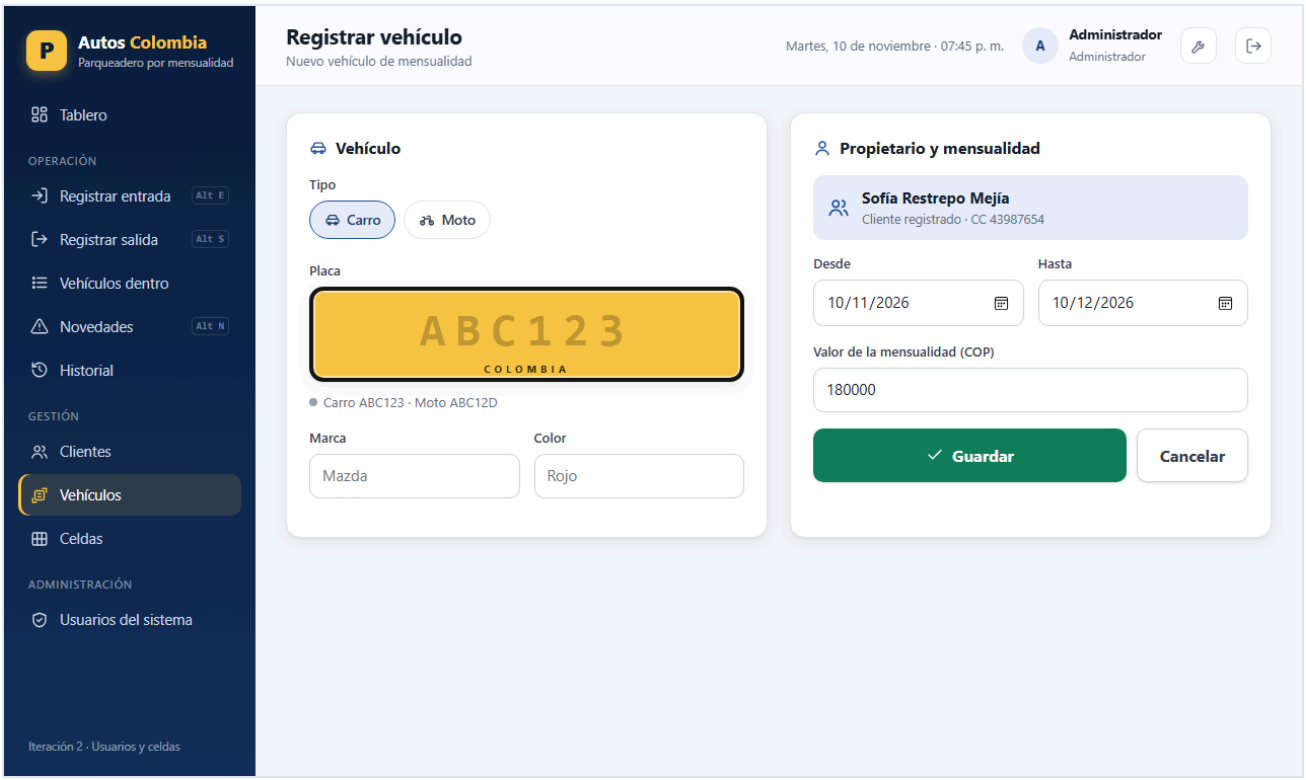


Figura 11. Asociar un vehículo de mensualidad a un cliente existente (HU18).



Figura 12. Gestión de celdas: resumen por tipo, mapa y listado (HU19, HU23).

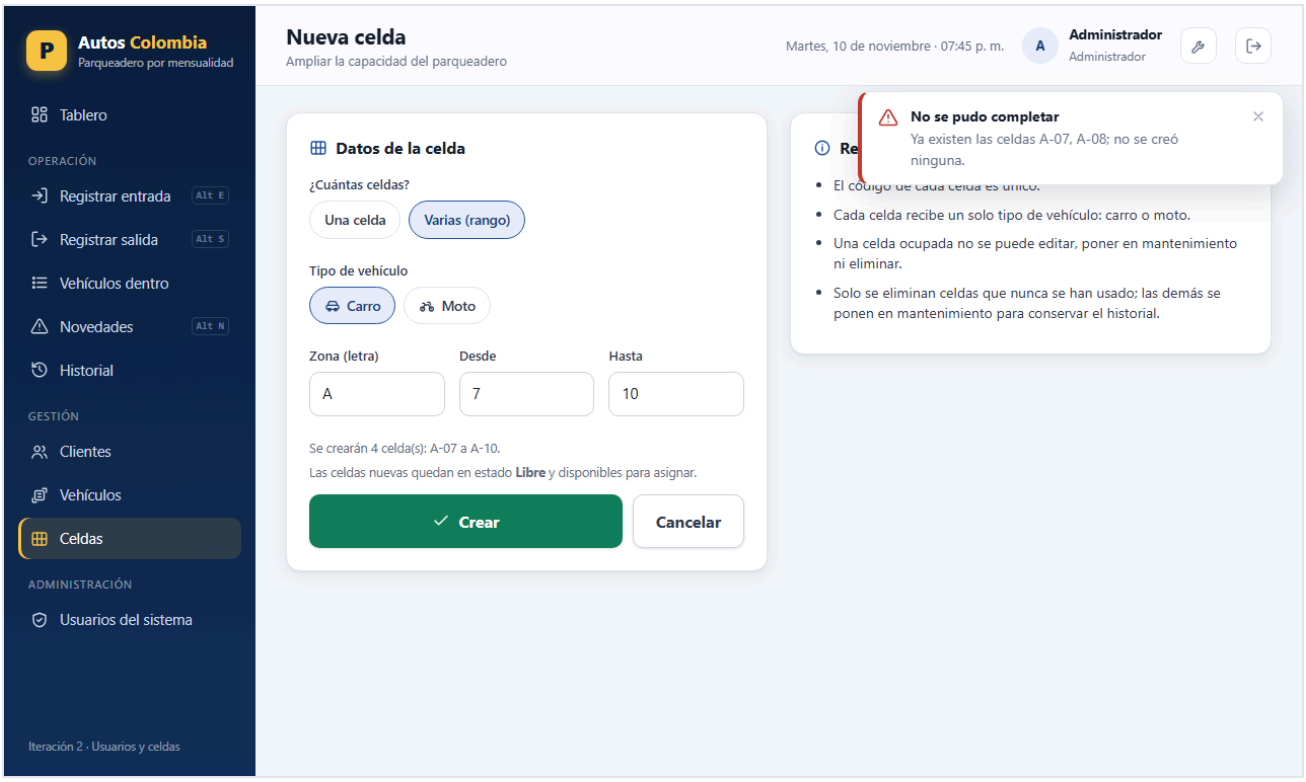


Figura 13. Crear celdas por rango (rechazo por códigos repetidos) (HU19).

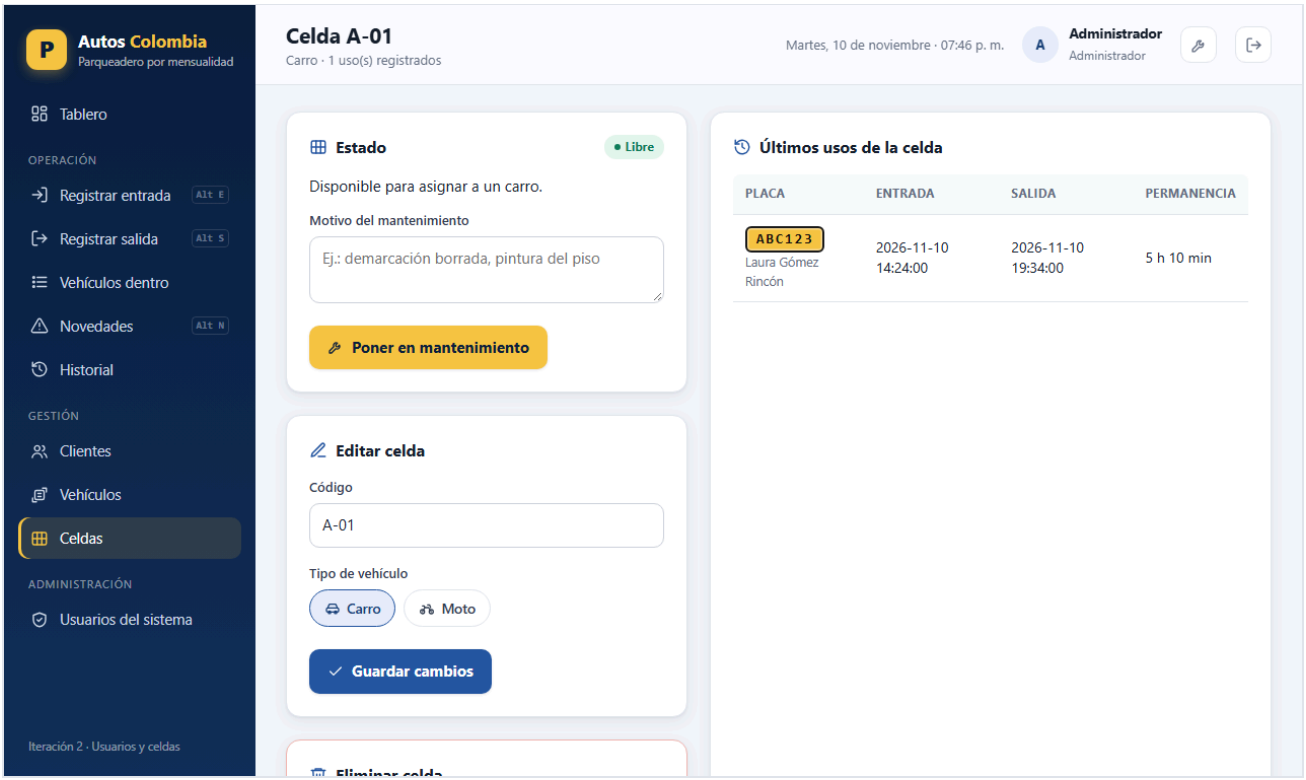


Figura 14. Detalle de celda: poner en mantenimiento, editar e historial de uso (HU20, HU21, HU23).

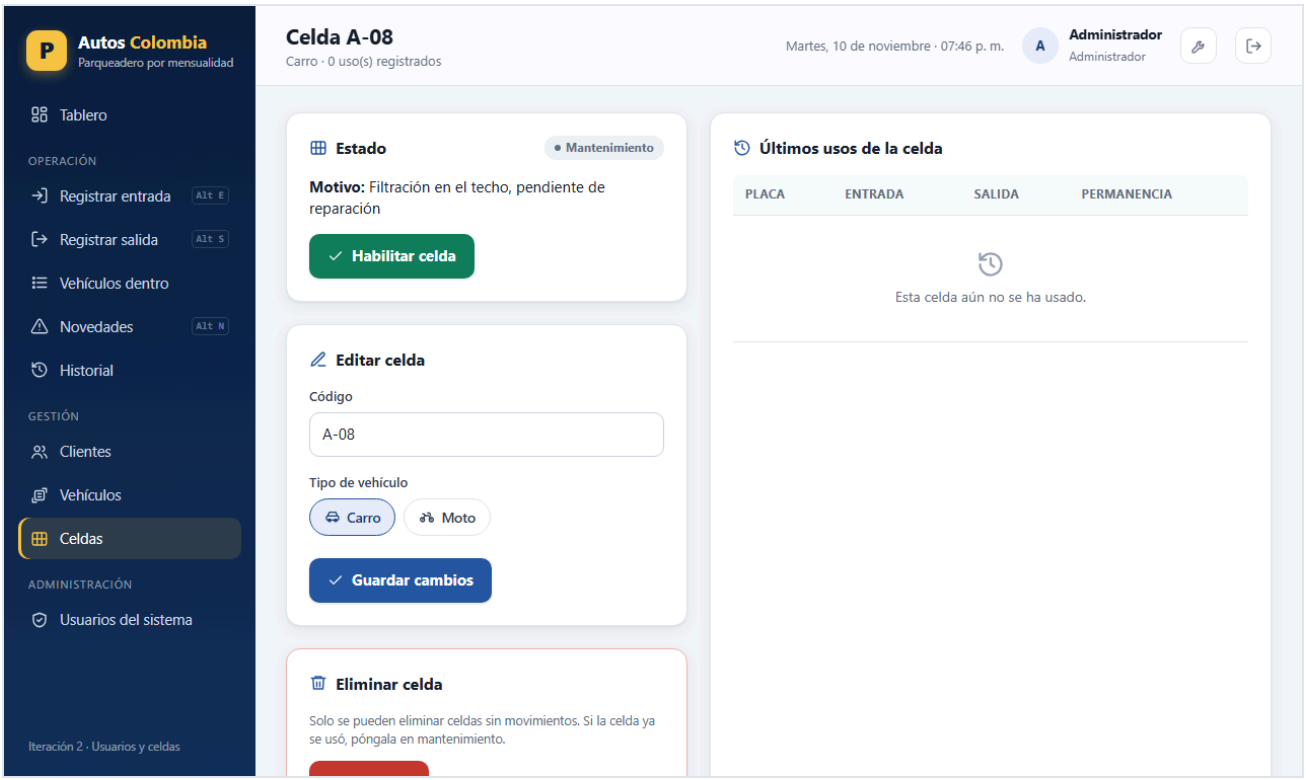


Figura 15. Celda en mantenimiento con su motivo y opción de habilitar (HU21).

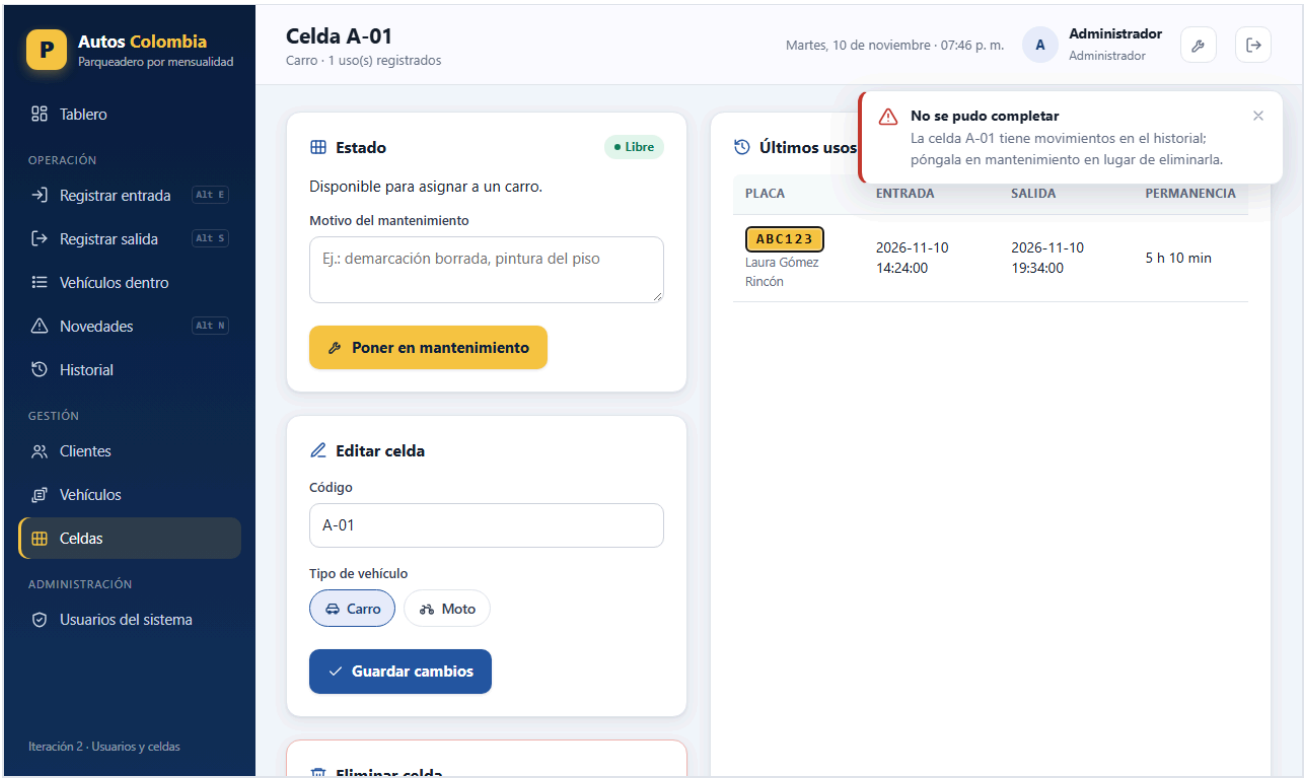


Figura 16. Eliminación rechazada: la celda tiene historial (HU22).

2.5 Diagramas de secuencia de las iteraciones 1 y 2

Los diagramas de secuencia se elaboraron en StarUML y muestran la colaboración entre las capas de la arquitectura: el actor envía una petición HTTP al **controlador** (Blueprint de Flask), este invoca al **servicio** que aplica las reglas de negocio y el servicio usa los **repositorios**, que ejecutan las consultas SQL sobre SQLite. Se usan fragmentos **alt** (alternativas), **opt** (opcional), **loop** (repetición) y **break** (interrupción por error). Las flechas punteadas son respuestas.

Diagrama	Iteración	Escenario	Casos de uso / historias
SD01	1	Iniciar sesión (incluye la validación de usuario activo de la iteración 2)	UC01 · HU01
SD02	1	Registrar entrada de vehículo (incluye la validación de cliente activo)	UC02 · HU02, HU03
SD03	1	Registrar salida de vehículo	UC03 · HU04
SD04	1	Registrar novedad	UC04 · HU05
SD05	2	Crear usuario del sistema	UC10 · HU10
SD06	2	Activar o desactivar usuario	UC12 · HU12
SD07	2	Registrar cliente y asociar vehículo	UC16, UC20 · HU15, HU18
SD08	2	Crear celdas por rango	UC24 · HU19
SD09	2	Mantenimiento y habilitación de celda	UC26, UC27 · HU21
SD10	2	Eliminar celda	UC28 · HU22

Iteración 1

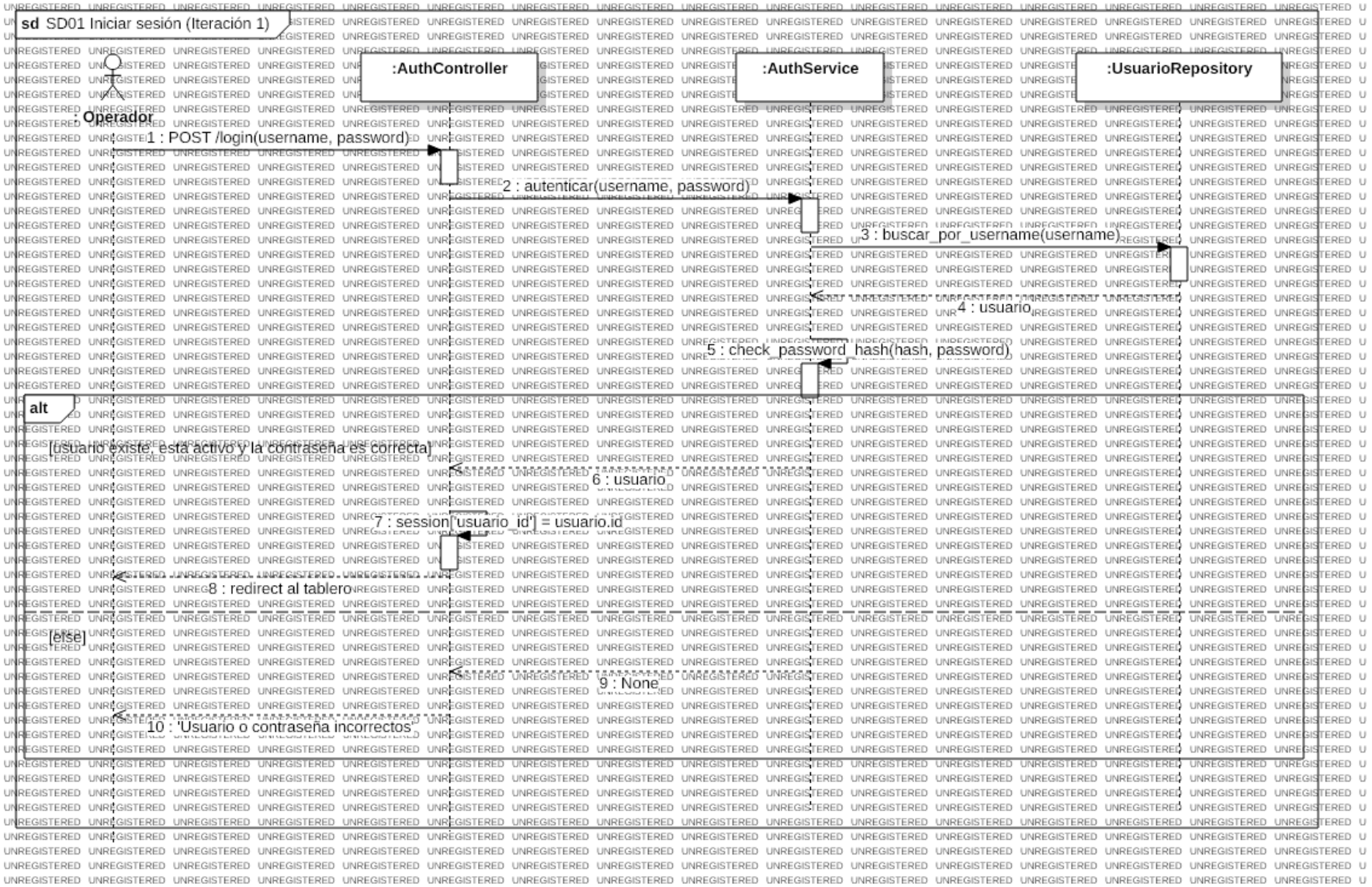


Figura 17. SD01 · Iniciar sesión.

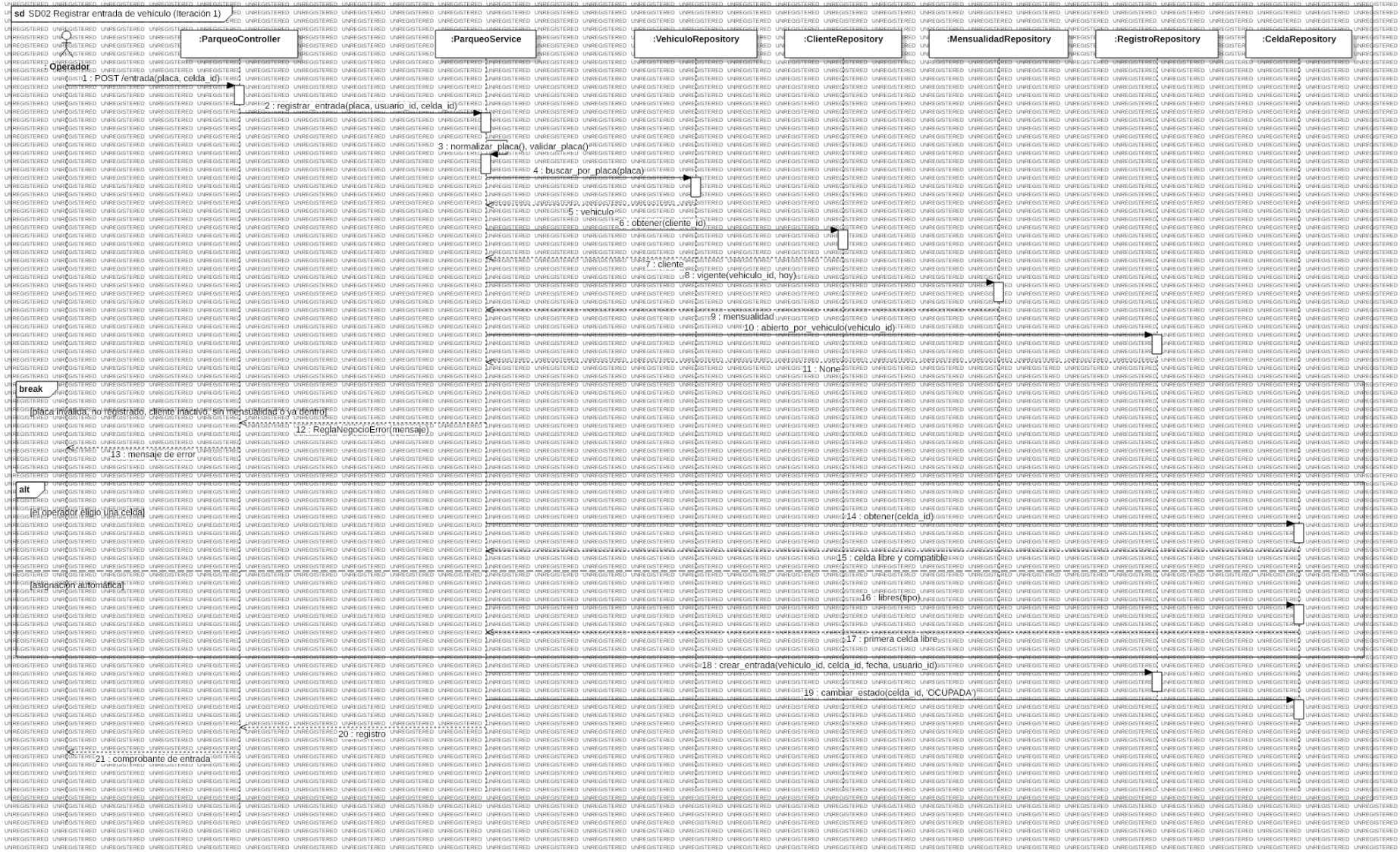


Figura 18. SD02 · Registrar entrada de vehículo.

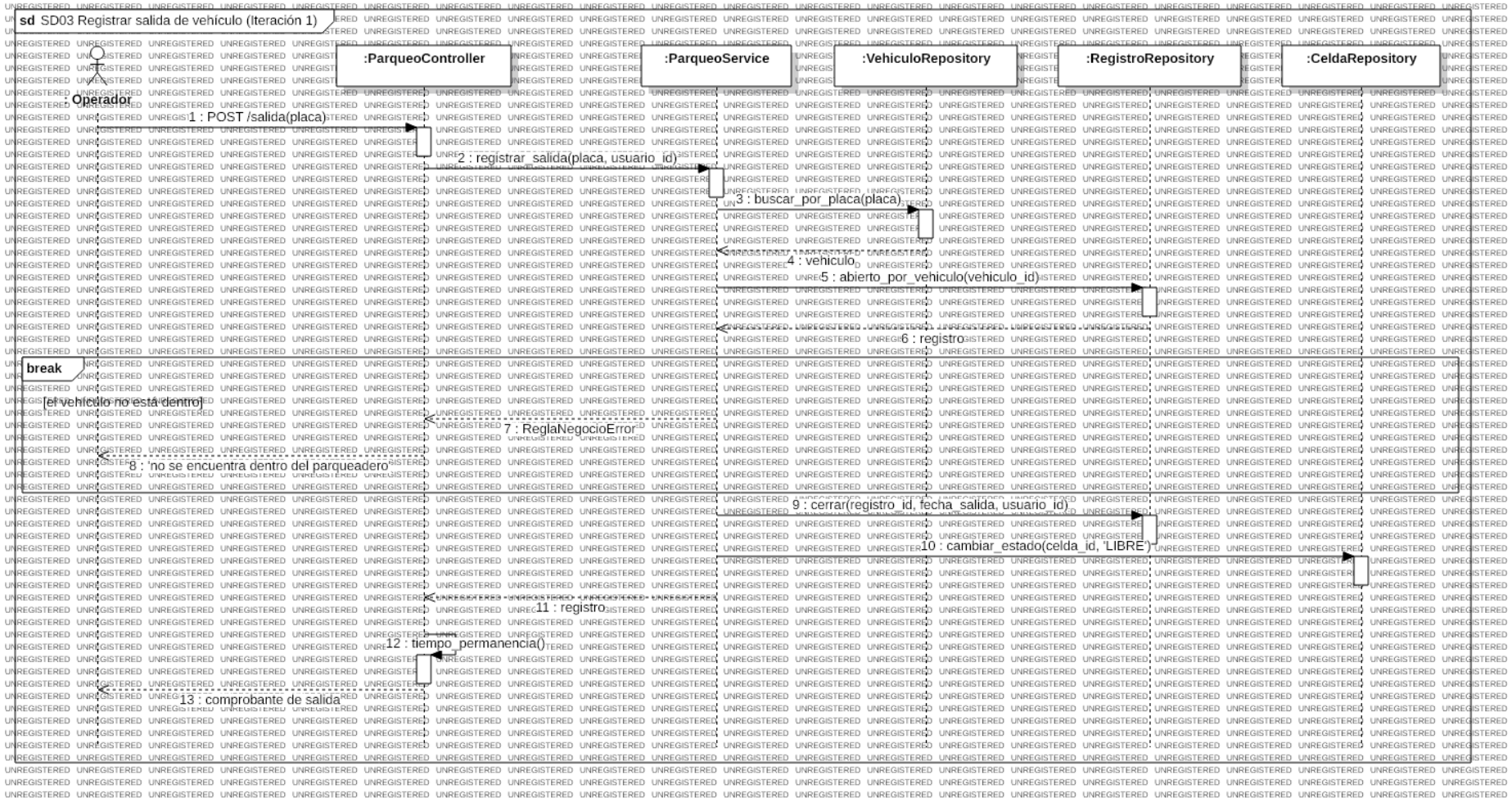


Figura 19. SD03 · Registrar salida de vehículo.

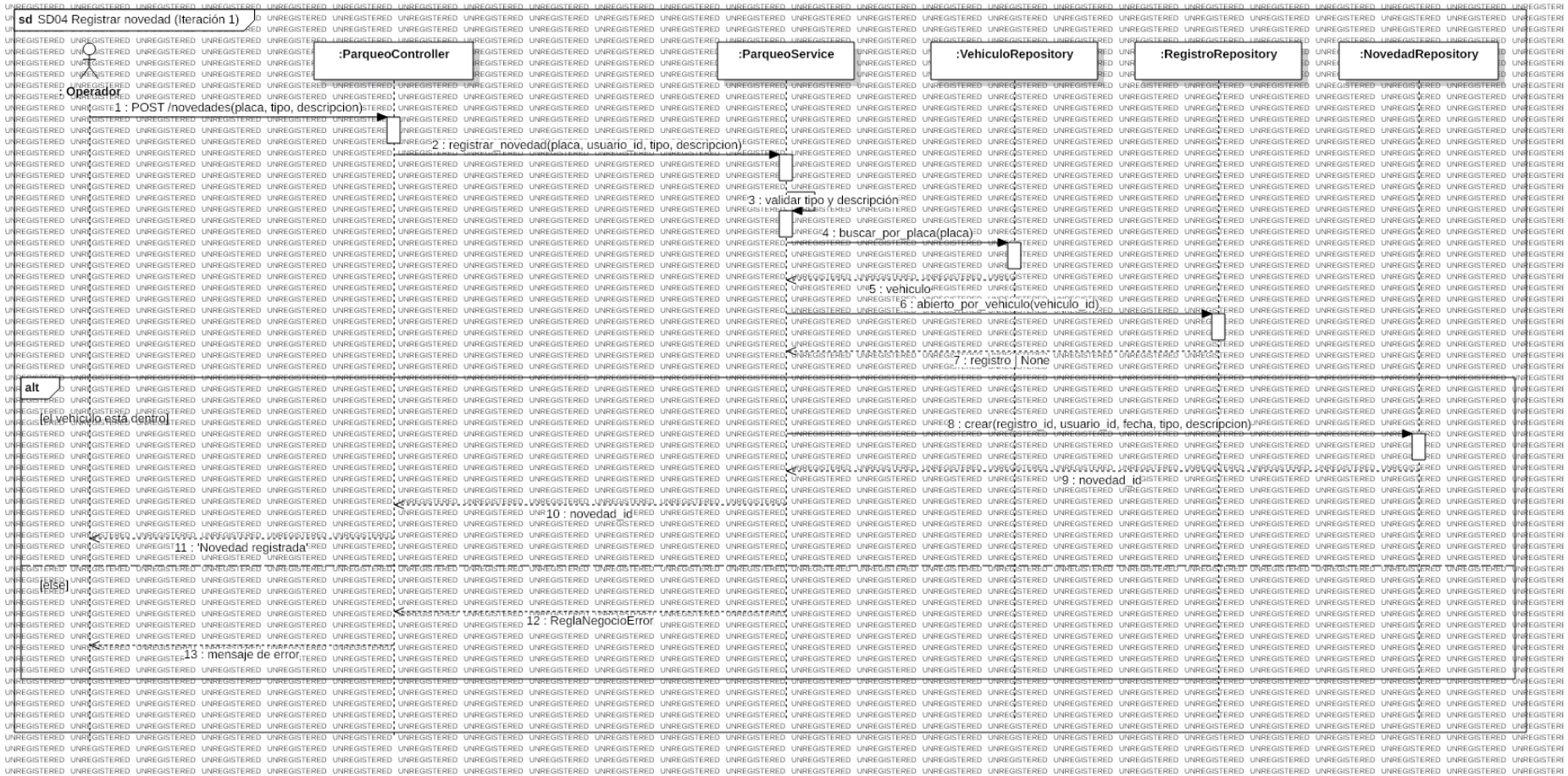


Figura 20. SD04 · Registrar novedad.

Iteración 2

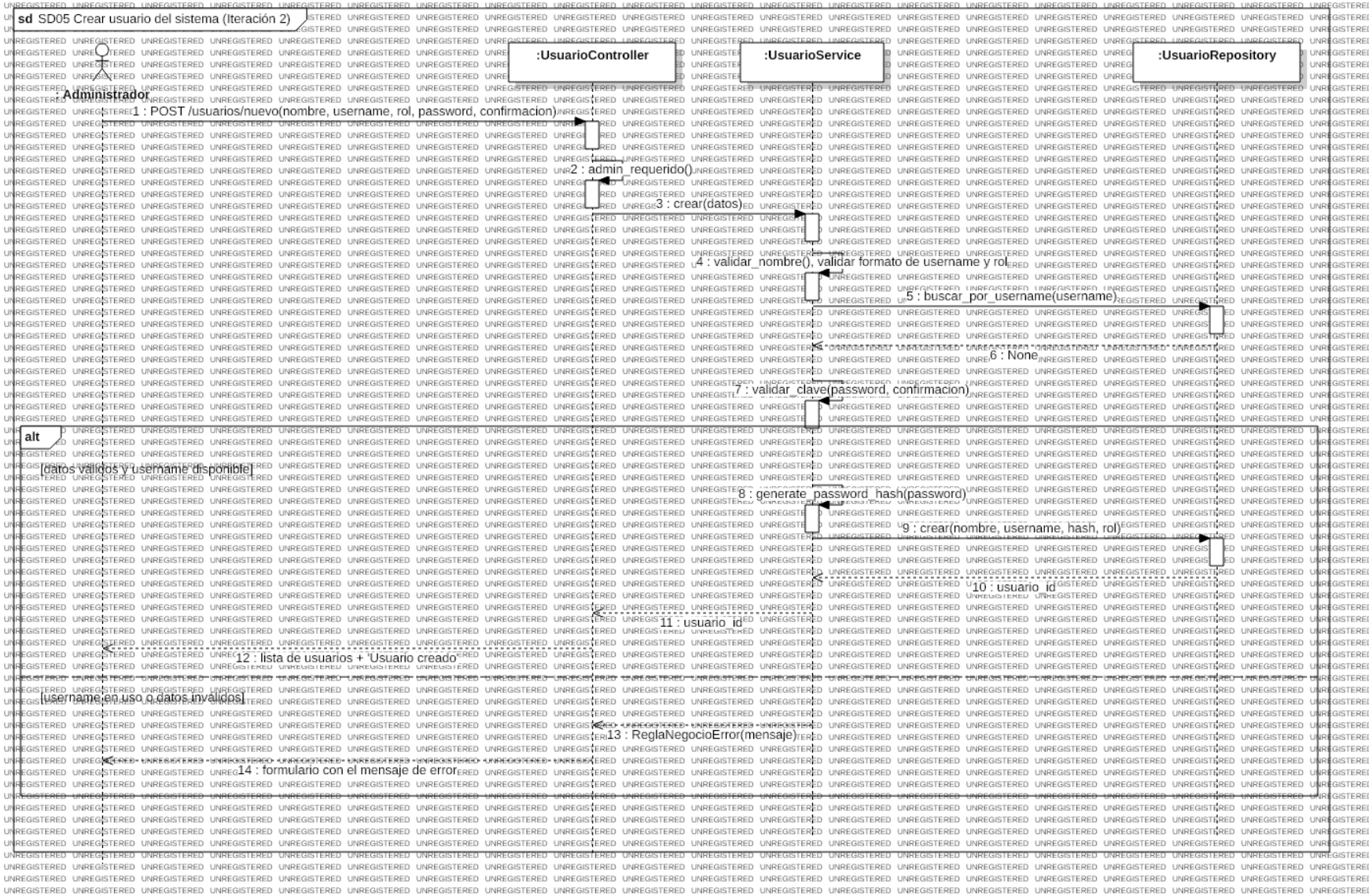


Figura 21. SD05 · Crear usuario del sistema.

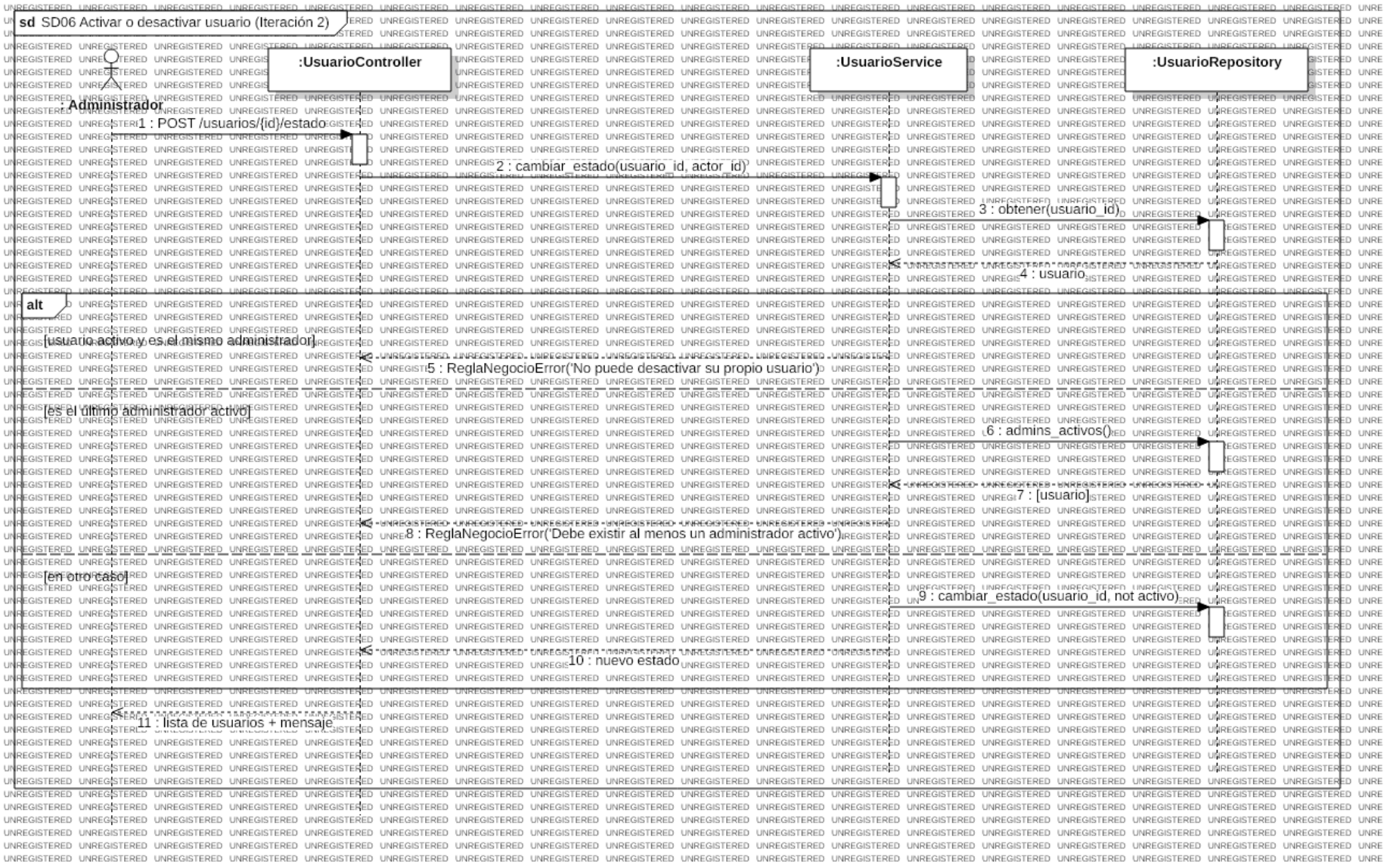


Figura 22. SD06 · Activar o desactivar usuario.

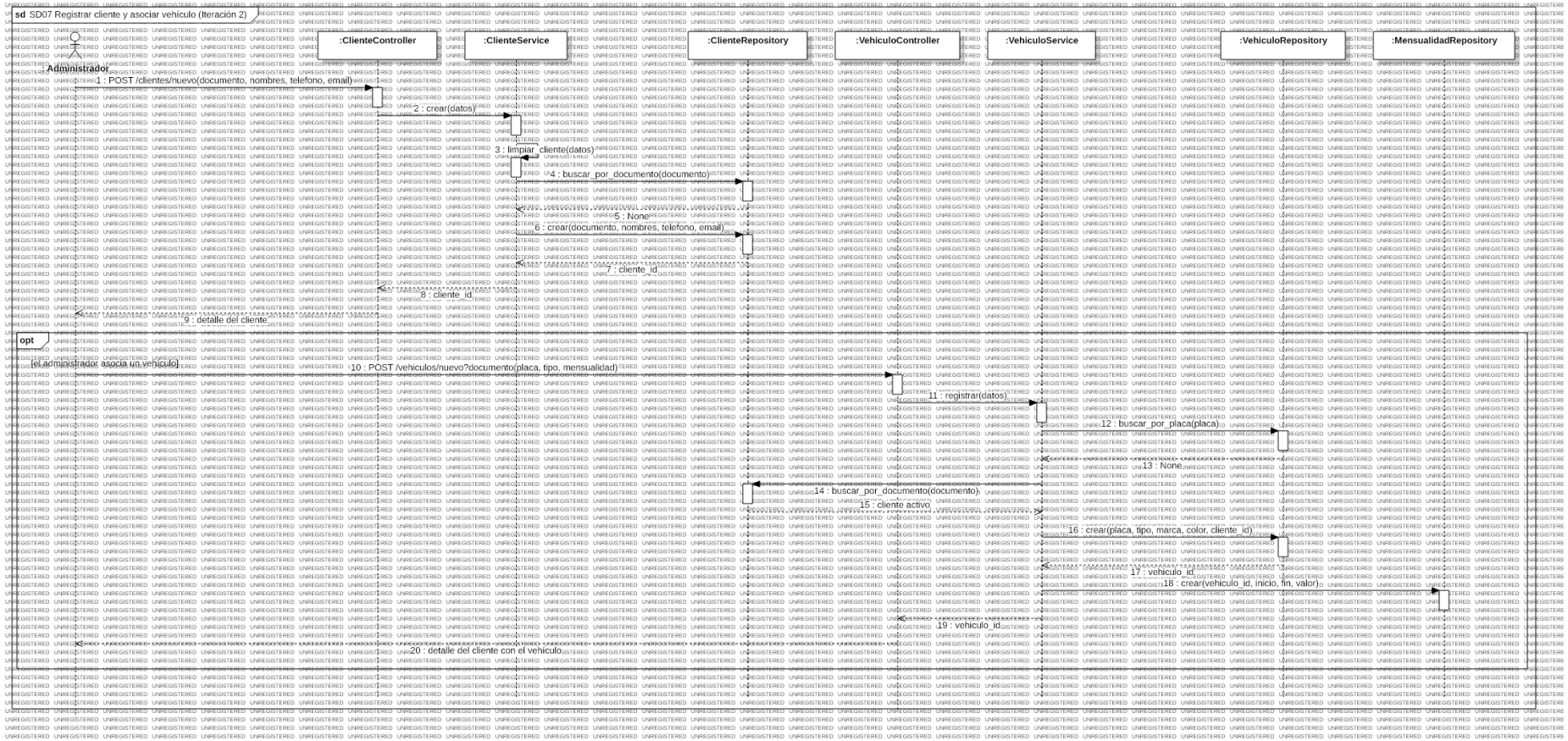


Figura 23. SD07 · Registrar cliente y asociar vehículo.

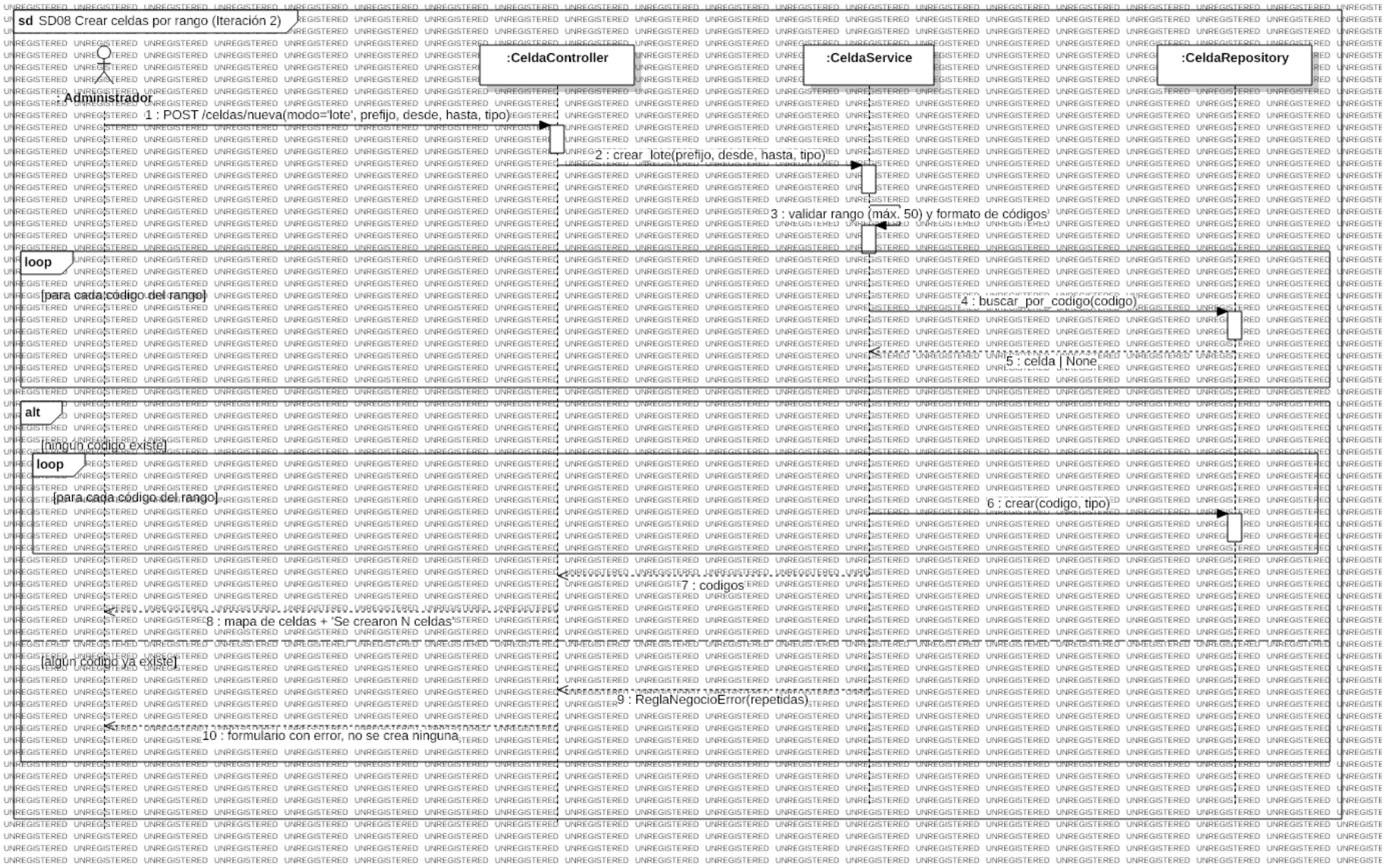


Figura 24. SD08 · Crear celdas por rango.

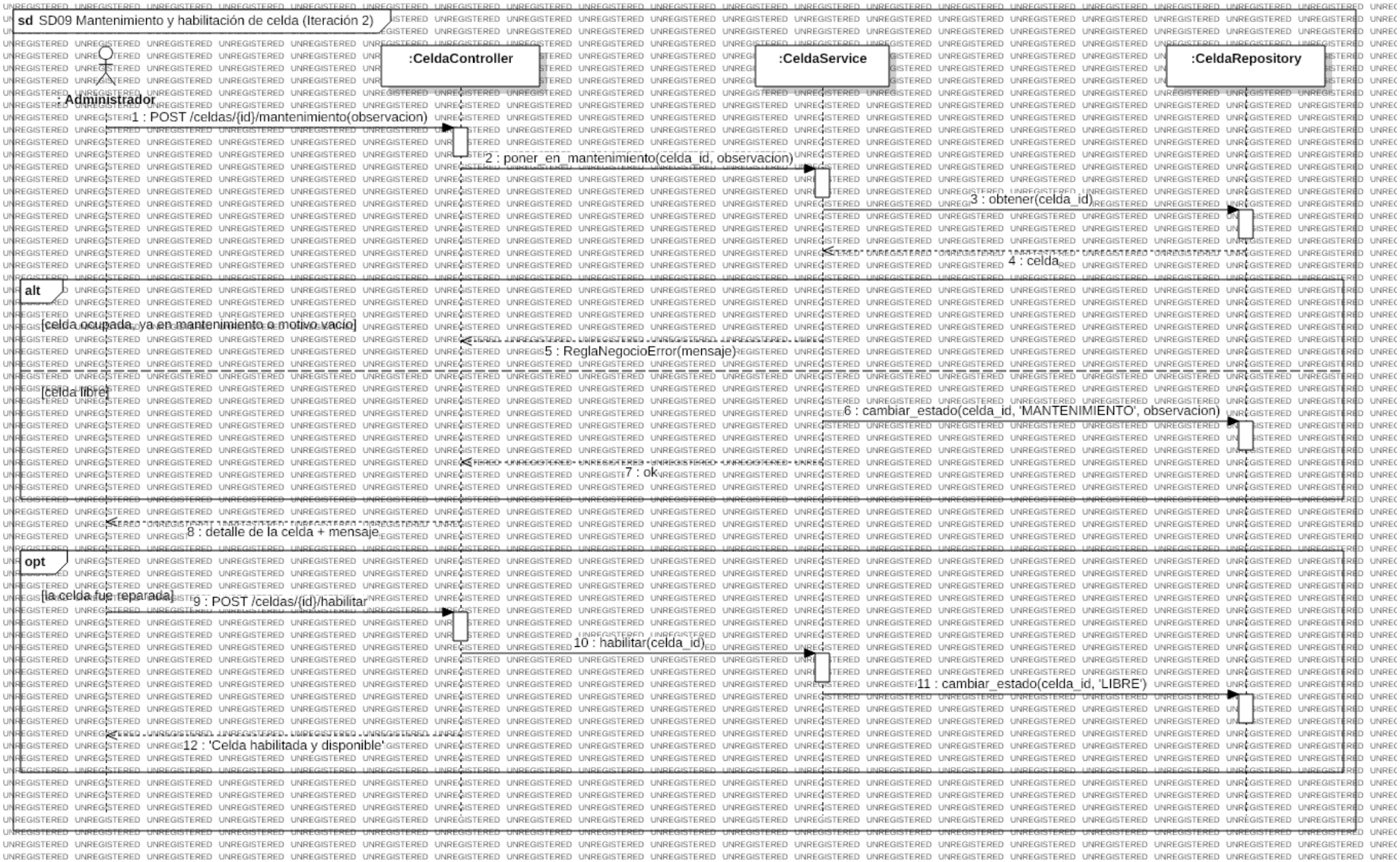


Figura 25. SD09 · Poner celda en mantenimiento y habilitarla.

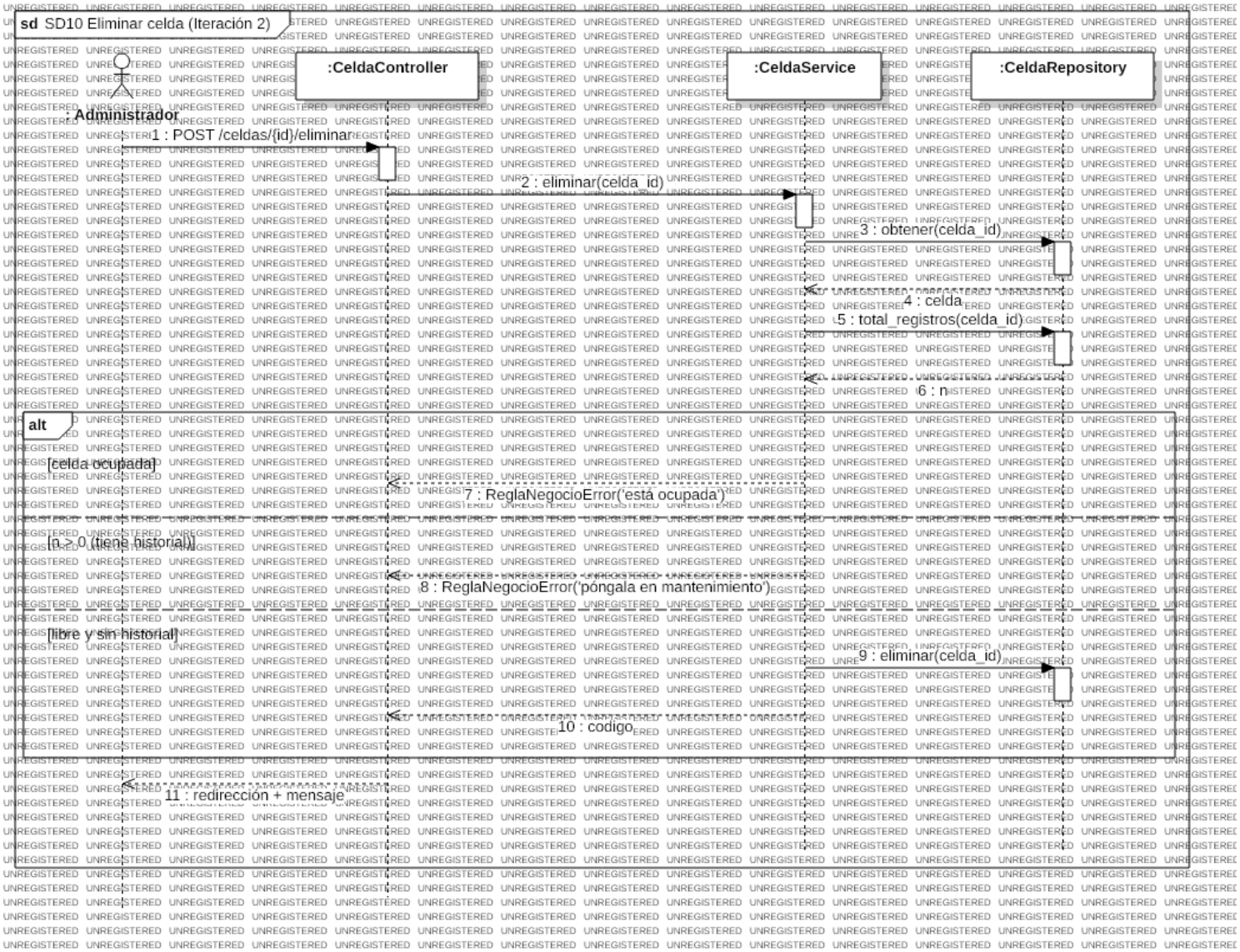


Figura 26. SD10 · Eliminar celda.

2.6 Diagrama de componentes de las iteraciones 1 y 2

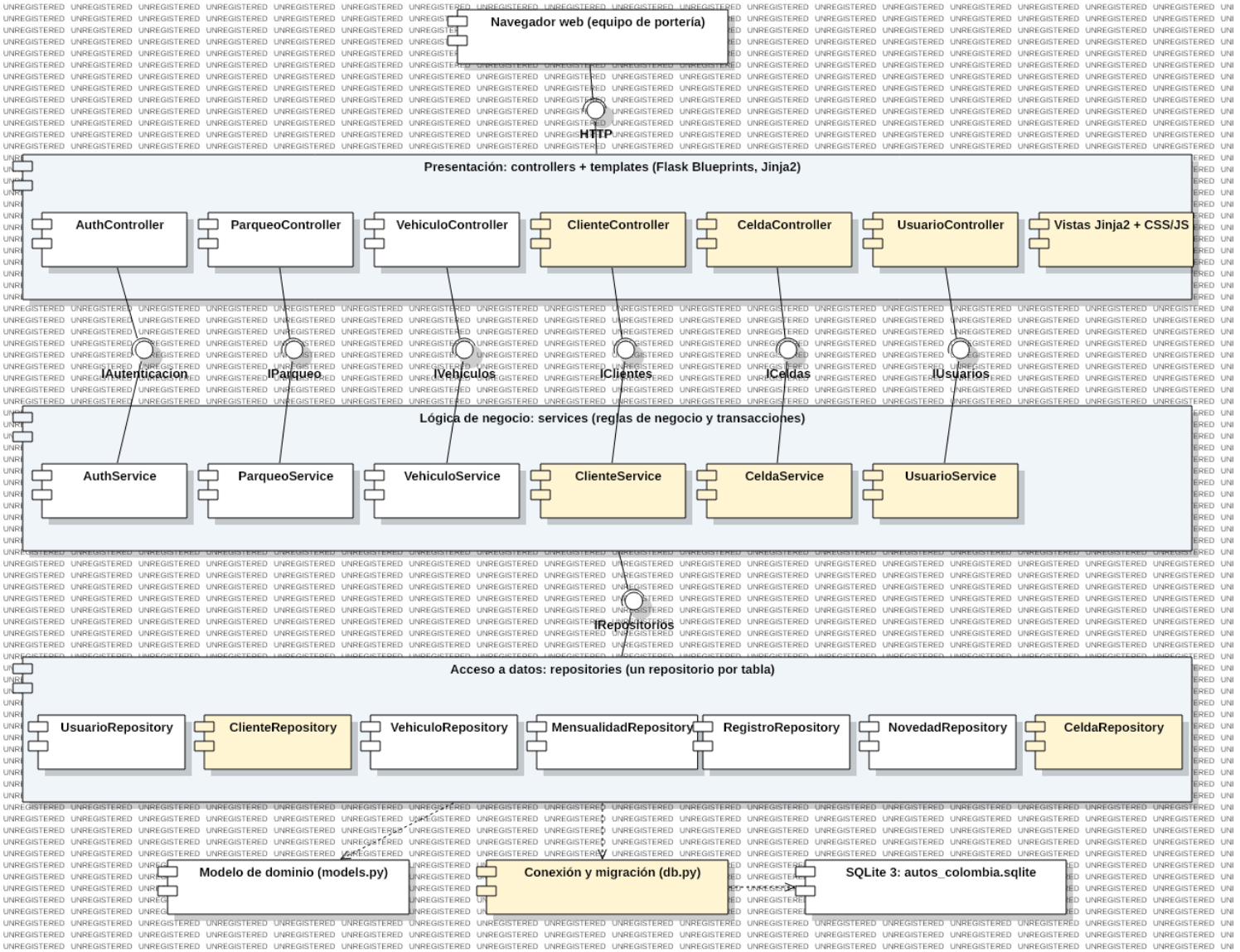


Figura 27. Diagrama de componentes (StarUML). En amarillo, los componentes nuevos o ampliados en la iteración 2.

El sistema se organiza en tres subsistemas que se comunican mediante interfaces. La capa de **presentación** expone la interfaz HTTP al navegador del equipo de portería; cada controlador depende de la interfaz que ofrece su servicio (*IAutenticacion, IParqueo, IVehiculos, IClientes, ICeldas, IUsuarios*), de modo que los controladores no conocen la base de datos. La **lógica de negocio** usa la interfaz *IRepositorios* de la capa de **acceso a datos**, que trabaja con el modelo de dominio y con la conexión SQLite.

Componente	Responsabilidad	Iteración
AuthController / AuthService	Inicio y cierre de sesión, control de acceso por rol; rechaza usuarios inactivos.	1 (ajustado en 2)
ParqueoController / ParqueoService	Entradas, salidas, novedades, tablero e historial; valida además que el cliente esté activo.	1 (ajustado en 2)
VehiculoController / VehiculoService	Registro de vehículos de mensualidad; ahora también desde el detalle de un cliente.	1 (ajustado en 2)
UsuarioController / UsuarioService	Gestión de usuarios del sistema y contraseñas.	2
ClienteController / ClienteService	Gestión de clientes y su estado.	2
CeldaController / CeldaService	Creación, edición, mantenimiento, habilitación, eliminación y consulta de celdas.	2
Repositorios	Un repositorio por tabla; `ClienteRepository` se separó y `CeldaRepository` se amplió.	1 y 2
db.py	Conexión, creación del esquema, datos de ejemplo y migración automática de la base de datos de la iteración 1.	1 (ampliado en 2)

2.7 Plan de pruebas resumido · Gestión de usuarios y gestión de celdas

Proyecto	Autos Colombia · Sistema de gestión de parqueadero por mensualidad
Iteración / versión	Iteración 2 · Gestión de usuarios y gestión de celdas
Responsable	David Santiago Cantor Torres
Fecha de ejecución	10 de noviembre de 2026

1. Objetivo y alcance

Verificar que las funciones de gestión de usuarios del sistema, de clientes y de celdas cumplen los requerimientos RF16–RF35, las reglas RN05–RN10 y los requerimientos no funcionales de seguridad e integridad, y que la iteración 1 sigue funcionando. **Incluye:** reglas de negocio de los servicios, pantallas y rutas web, control de acceso por rol y migración de datos. **No incluye:** pagos (iteración 3), pruebas de carga ni compatibilidad con navegadores antiguos.

2. Estrategia

Tipo de prueba	Cómo se realiza
Unitarias de servicio	Cada regla de negocio se prueba sobre `UsuarioService`, `ClienteService` y `CeldaService` con una base de datos SQLite en memoria (`unittest`).
Integración web	Con el cliente de pruebas de Flask se recorren las pantallas y se envían formularios como administrador y como operador.
Seguridad	Acceso por URL a funciones administrativas con un operador; inicio de sesión de un usuario desactivado.
Regresión	Se ejecutan de nuevo las 15 pruebas de la iteración 1 (entradas, salidas, novedades).
Exploratoria	Revisión visual de las pantallas en Microsoft Edge (capturas de la sección 2.4).

3. Ambiente y datos de prueba

Windows 11, Python 3.13, Flask 3.1.2, SQLite 3, Microsoft Edge. Datos de ejemplo de `database/seed.sql`: 5 clientes, 5 vehículos (QRS234 con mensualidad vencida), 12 celdas (A-08 en mantenimiento) y 4 usuarios: `admin`, `operador`, `ltorres` (operadora) y `mdiaz` (inactivo).

4. Criterios

Entrada: código integrado y base de datos de ejemplo cargada. **Aceptación:** el 100 % de los casos de prioridad alta aprobados, ninguna falla en la regresión de la iteración 1 y ningún defecto abierto de severidad alta. **Suspensión:** si la aplicación no inicia o la base de datos no se puede migrar.

5. Casos de prueba y resultados

ID	HU / requisito	Caso y precondición	Datos y pasos	Resultado esperado	Resultado obtenido	Estado
CP-01	HU10 · RF16	**Crear usuario del sistema** Sesión de administrador.	Nombre «Ana María Ruiz», usuario «ARuiz», rol Operador, clave «Clave2026» (dos veces).	Se crea el usuario «aruiz» (en minúsculas), activo; la clave se guarda cifrada y permite iniciar sesión.	Igual al esperado `test_crear_usuario_y_autenticar`	**Aprobado**
CP-02	HU10 · RF17	**Usuario duplicado** Existe el usuario «operador».	Crear otro usuario con nombre de usuario «operador».	Se rechaza: «El nombre de usuario 'operador' ya está en uso». No se crea nada.	Igual al esperado `test_username_duplicado`	**Aprobado**
CP-03	HU10 · RF17	**Validar formato de usuario, contraseña y rol** Sesión de administrador.	Usuario «a b»; clave «corta1»; clave sin números; confirmación distinta; rol «SUPERVISOR».	Cada caso se rechaza con un mensaje que explica la regla incumplida.	Igual al esperado `test_username_y_clave_invalidos`	**Aprobado**
CP-04	HU11 · RF18	**Editar nombre y rol** Existe el operador «operador».	Cambiar nombre a «Pedro Pablo Operador» y rol a Administrador.	Se guardan el nuevo nombre y el rol; el nombre de usuario no cambia.	Igual al esperado `test_editar_nombre_y_rol`	**Aprobado**
CP-05	HU12 · RF19	**Usuario inactivo no inicia sesión** Usuario «operador» activo.	Desactivarlo e intentar iniciar sesión; luego activarlo e intentar de nuevo.	Inactivo: «Usuario o contraseña incorrectos». Activo de nuevo: ingresa normalmente.	Igual al esperado `test_usuario_desactivado_no_entra`, `test_usuario_inactivo_no_inicia_sesion`	**Aprobado**
CP-06	HU11, HU12 · RF20	**Proteger a los administradores** Un solo administrador activo.	Desactivarse a sí mismo; quitarse el rol; quitar el rol o desactivar al último administrador.	Todas las acciones se rechazan; siempre queda al menos un administrador activo.	Igual al esperado `test_no_se_desactiva_a_si_mismo_ni_al_ultimo_admin`	**Aprobado**
CP-07	HU13, HU14 · RF21, RF22	**Restablecer y cambiar contraseña** Usuario «operador».	Admin restablece a «NuevaClave1»; el usuario la cambia con clave actual errada, igual y luego válida.	Funciona la clave restablecida; se rechaza clave actual errada o repetida; tras el cambio solo sirve la nueva.	Igual al esperado `test_restablecer_y_cambiar_clave`	**Aprobado**
CP-08	HU15 · RF24, RF25	**Registrar cliente** Sesión de administrador.	Documento «1.098.765.432», nombre « Juan Pérez », tel. «300 123 4567», correo «Juan@Correo.COM».	Se guarda normalizado: 1098765432, «Juan Pérez», 3001234567, juan@correo.com; estado activo.	Igual al esperado `test_registrar_cliente_normaliza_datos`	**Aprobado**

CP-09	HU15 · RF25	**Validar datos del cliente** Existe el documento 1020304050.	Documento repetido; documento «12AB»; nombre de 1 letra; teléfono «12»; correo «juan@».	Cada registro se rechaza con el mensaje correspondiente.	Igual al esperado `test_validaciones_de_cliente`	**Aprobado**
CP-10	HU16 · RF26	**Editar cliente sin duplicar documento** Clientes 1 y 2 registrados.	Cambiar el teléfono del cliente 1; luego asignarle el documento del cliente 2.	El teléfono se actualiza; el documento duplicado se rechaza.	Igual al esperado `test_editar_cliente_sin_duplicar_documento`	**Aprobado**
CP-11	HU17 · RF28	**Cliente inactivo no ingresa vehículos** Cliente con vehículo ABC123 y mensualidad vigente.	Desactivar el cliente; registrar entrada de ABC123 y asociarle un vehículo; reactivarlo.	Inactivo: entrada y nuevo vehículo rechazados. Reactivado: la entrada se registra.	Igual al esperado `test_cliente_inactivo_no_ingresa_vehiculos`	**Aprobado**
CP-12	HU17 · RF28	**No desactivar cliente con vehículo dentro** ABC123 está dentro del parqueadero.	Desactivar al propietario de ABC123.	Se rechaza: debe registrarse primero la salida.	Igual al esperado `test_no_desactiva_cliente_con_vehiculo_dentro`	**Aprobado**
CP-13	HU18 · RF29	**Asociar vehículo a cliente** Cliente 55667788 sin vehículos.	Registrar la moto WER45T con mensualidad desde el detalle del cliente.	El vehículo queda asociado al cliente y aparece en su detalle.	Igual al esperado `test_asociar_vehiculo_a_cliente_existente`	**Aprobado**
CP-14	HU19 · RF30	**Crear una celda** Sesión de administrador.	Código « a-09 » tipo Carro; luego «A-01» (existente), «CELDA9» y tipo «BUS».	Se crea A-09 libre; los demás casos se rechazan (código repetido, formato, tipo).	Igual al esperado `test_crear_celda`	**Aprobado**
CP-15	HU19 · RF31	**Crear celdas por rango** Existen M-01 a M-04.	Crear M-05 a M-07; luego M-07 a M-09; luego B-01 a B-60.	Se crean 3 celdas; el segundo rango no crea ninguna (M-07 repetida); el tercero supera el máximo de 50.	Igual al esperado `test_crear_celdas_por_rango`	**Aprobado**
CP-16	HU20 · RF32	**Editar celda** A-01 libre; JKL456 ocupa otra celda.	Cambiar A-01 a «A-11»/Moto; usar el código «A-02»; editar la celda ocupada.	Primer cambio guardado; código repetido y celda ocupada se rechazan.	Igual al esperado `test_editar_celda`	**Aprobado**
CP-17	HU21 · RF33	**Mantenimiento y habilitación** A-01 libre.	Ponerla en mantenimiento con motivo «Demarcación borrada»; intentar asignarla; habilitarla; motivo vacío.	Queda en mantenimiento con su motivo y no se asigna; al habilitar vuelve a LIBRE; sin motivo se rechaza.	Igual al esperado `test_mantenimiento_y_habilitar`	**Aprobado**

CP-18	HU21 · RN03	**No mantenimiento de celda ocupada** ABC123 ocupa una celda.	Poner esa celda en mantenimiento.	Se rechaza: la celda está ocupada.	Igual al esperado `test_no_mantenimiento_de_celda_ocupada`	**Aprobado**
CP-19	HU22 · RF34	**Eliminar celda** Celda A-30 nueva; celda usada por ABC123.	Eliminar A-30; eliminar la celda ocupada; registrar salida y eliminarla.	A-30 se elimina; la ocupada y la que tiene historial se rechazan (se sugiere mantenimiento).	Igual al esperado `test_eliminar_celda`	**Aprobado**
CP-20	RNF10	**Control de acceso por rol** Sesión de operador.	Abrir /usuarios, /usuarios/nuevo, /clientes/nuevo, /celdas/nueva, editar cliente, eliminar celda.	Se redirige al tablero sin ejecutar; consultas de clientes y celdas y «mi contraseña» sí están permitidas.	Igual al esperado `test_operador_no_accede_a_administracion`	**Aprobado**
CP-21	HU10, HU15, HU19, HU21	**Flujo web del administrador** Sesión de administrador.	Recorrer todas las pantallas; crear usuario, cliente, celdas B-01 a B-03 y poner A-01 en mantenimiento.	Todas las pantallas responden 200 y cada operación muestra su confirmación.	Igual al esperado `test_admin_navega_y_gestiona`	**Aprobado**

6. Resumen de la ejecución

21/21	**22**	**15/15**	**0**
casos de prueba aprobados	pruebas automatizadas nuevas	pruebas de regresión iteración 1	fallas en la última ejecución

Comando: `python -m unittest discover -s tests -t . -v` · 37 pruebas, todas aprobadas.

7. Defectos encontrados

ID	Descripción	Severidad	Estado
DEF-01	La prueba de regresión `test_salida_libera_celda_y_calcula_permanencia` usaba una fecha fija (20/10/2026) mientras las mensualidades de ejemplo son relativas a la fecha actual; al pasar los días la entrada se rechazaba por mensualidad vencida.	Media (prueba)	Corregido: la prueba usa la fecha del día.

Se verificó además la migración (RNF15): una base de datos creada con el esquema de la iteración 1 se abrió con la versión nueva; las columnas `cliente.activo`, `celda.observacion` y `usuario.creado_en` se agregaron automáticamente, todas las pantallas respondieron correctamente y la desactivación de un cliente se guardó sin perder datos.

2.8 Código fuente y repositorio

Repositorio GitHub: https://github.com/Santiago071007/Iteraci-n-1

El repositorio contiene el código de las dos iteraciones, la base de datos de ejemplo, las pruebas, el proyecto StarUML (`docs/AutosColombia.mdj`), las imágenes de los diagramas y mockups, y los generadores de esta documentación.

Cambios de la iteración 2

Archivo	Cambio
`services/usuario_service.py`	Nuevo. Crear, editar, activar/desactivar, restablecer y cambiar contraseña; política de contraseñas y protección del último administrador.
`services/cliente_service.py`	Nuevo. Validación y normalización de clientes, edición y cambio de estado.
`services/celda_service.py`	Nuevo. Crear (una o por rango), editar, mantenimiento con motivo, habilitar y eliminar celdas.
`services/parqueo_service.py`	La entrada rechaza vehículos de clientes inactivos; el mantenimiento de celdas pasó a `CeldaService`.
`controllers/usuario_controller.py`, `cliente_controller.py`, `celda_controller.py`	Rutas nuevas protegidas por rol (`/usuarios`, `/clientes`, `/celdas`).
`repositories/`	`ClienteRepository` en archivo propio; nuevos métodos en `UsuarioRepository` y `CeldaRepository`.
`templates/usuarios`, `clientes`, `celdas`	Vistas nuevas; menú con secciones Gestión y Administración.
`database/schema.sql`, `db.py`	Columnas `usuario.creado_en`, `cliente.activo` y `celda.observacion`; migración automática.
`tests/test_usuarios_celdas.py`	Nuevo. 22 pruebas que automatizan los casos CP-01 a CP-21.

Ejecución

```
python -m venv .venv
.venv\Scripts\activate          (Windows)   |   source .venv/bin/activate   (Linux/Mac)
pip install -r requirements.txt
flask --app app run             → abrir http://127.0.0.1:5000
Usuarios de prueba:  admin / admin123   ·   operador / operador123   ·   ltorres / Turno2026
```

Fragmento: protección de administradores (`services/usuario_service.py`)

```
    def cambiar_estado(self, usuario_id: int, actor_id: int) -> bool:
        """Activa o desactiva el usuario. Devuelve el nuevo estado (True = activo)."""
        usuario = self.obtener(usuario_id)
        if usuario.activo:
            if usuario.id == actor_id:
                raise ReglaNegocioError("No puede desactivar su propio usuario.")
            if usuario.es_admin() and not self._queda_otro_admin(usuario.id):
                raise ReglaNegocioError("Debe existir al menos un administrador activo.")
        with self.db:
            self.usuarios.cambiar_estado(usuario.id, not usuario.activo)
        return not usuario.activo
```

Fragmento: eliminación de celdas (`services/celda_service.py`)

```
    def eliminar(self, celda_id: int) -> str:
        celda = self.obtener(celda_id)
        if celda.esta_ocupada():
            raise ReglaNegocioError(f"La celda {celda.codigo} está ocupada; no se puede eliminar.")
        if self.celdas.total_registros(celda.id):
            raise ReglaNegocioError(
                f"La celda {celda.codigo} tiene movimientos en el historial; póngala en mantenimiento en
lugar de eliminarla."
            )
        with self.db:
            self.celdas.eliminar(celda.id)
        return celda.codigo
```

3. Conclusiones

- La iteración 2 completa la administración del parqueadero: el administrador ya no depende de cambios en la base de datos para crear empleados, registrar clientes o ampliar el número de celdas, y el operador tiene acceso de consulta a la información que necesita en portería.
- Separar los usuarios del sistema (quién opera) de los clientes (quién recibe el servicio) aclaró las reglas de cada uno: los empleados se controlan con roles y contraseñas, y los clientes con su estado, que repercute directamente en la entrada de vehículos construida en la iteración 1.
- Desactivar en lugar de borrar, y eliminar celdas solo cuando no tienen historial, preserva la trazabilidad lograda en la iteración 1: cada entrada, salida y novedad sigue apuntando a un usuario, un cliente y una celda existentes.
- Los diagramas de secuencia de ambas iteraciones confirmaron que la arquitectura en capas se mantuvo: todos los escenarios siguen el mismo recorrido controlador → servicio → repositorio, y el diagrama de componentes muestra que la iteración 2 agregó componentes sin modificar las interfaces existentes.
- El plan de pruebas, automatizado con 37 pruebas, permitió detectar y corregir una prueba de la iteración 1 dependiente de la fecha y verificar que las nuevas reglas no rompieron la operación existente. La base queda lista para la iteración 3 (pagos), que podrá apoyarse en los clientes y mensualidades ya gestionados.

4. Referencias

- Pallets Projects. *Flask Documentation (3.1.x)*. https://flask.palletsprojects.com/
- Pallets Projects. *Werkzeug: Utilities · Password hashing*. https://werkzeug.palletsprojects.com/
- MKLabs. *StarUML Documentation*. https://docs.staruml.io/
- Object Management Group. *Unified Modeling Language (UML) Specification, v2.5.1*. 2017.
- Cohn, M. *User Stories Applied: For Agile Software Development*. Addison-Wesley, 2004.
- Sommerville, I. *Ingeniería de Software*. 9.ª ed. Pearson, 2011.
- ISO/IEC/IEEE 29119-3:2021. *Software and systems engineering — Software testing — Part 3: Test documentation*.